claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_b3a30c42-346\agent-a3983c9041232c15e.jsonl`
- SHA-256: `a8f1cdf6b1e54f542c90f34e2825b88a3d9864b8fe568f96facafb590c56d675`
- Source modified: `2026-06-16T02:30:49+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_b3a30c42-346`
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`

Transcript

1. user (2026-06-16T02:28:01.392Z)

```
Propose a design optimized SPECIFICALLY for the user's stated core goal: a reviewer receiving a
refactor/small-change PR from someone on another machine (no video) must trust the suite to flag
side effects. Center on CHARACTERIZATION (golden-output snapshot) testing of the deterministic
pipeline from a committed seam, with a crisp, documented snapshot-UPDATE workflow so intentional
behavior changes are explicit and reviewable in the diff. Detail: the contributor and reviewer
UX end-to-end; how 'no side effects' is actually guaranteed AND its limits (what it does NOT
cover, e.g. detector-level or video-IO changes); repo-size & determinism handling (cross-OS,
Windows-vs-WSL/Linux, float, line endings) so snapshots are stable across machines; licensing-
safe artifact selection. Be explicitly honest about false-safety.

GROUNDING ? findings from codebase analysis (JSON):
[
  {
    "area": "Existing regression/golden/eval infrastructure in backend/eval (badminton rally
detector) ? for a \"video-free regression from committed structured files\" design",
    "summary": "The repo already has a mature, layered, mostly-pure eval stack under
backend/eval/ that is video-free at the SCORING boundary: pure interval metrics (metrics.py,
segmentation_metrics.py), one canonical GT CSV loader (gt_loader.py), a DB-read harness
(harness.py), a baseline-snapshot regression gate (regression.py + run_regression.py, exit
0/1/3) and a sibling eval harness CLI (run_eval.py), an A/B experiment engine (experiment.py)
with a committed leaderboard, an ablation driver (ablation.py), and two offline replay-substrate
producers (fusion_golden.py -> per-video *_golden_features.json; export_wasb_segments.py).
HOWEVER, none of the existing flows runs from committed-in-repo structured files end-to-end. Two
distinct \"sources of truth\" exist: (a) the DB path (harness/regression read predictions from a
SQLite DB that only the GPU pipeline can populate), and (b) the trajectory/feature path
(calibrate_wasb, distill_local, fusion_golden, served_gate_a, ablation read WASB trajectory CSVs
/ golden_features JSON / .rallies.csv whose paths come from output/human_lovo_manifest.json ?
which is gitignored and points at absolute paths OUTSIDE the repo). ablation.py is the closest
existing thing to a video-free, file-driven regression: it loads *_golden_features.json from a
directory and runs entirely on CPU with no DB and no video ? but those JSONs are not committed.
The committed structured artifacts that DO survive a clean checkout live in eval_baselines/
(tracked): heuristic_lovo_n6.json (the LOVO floor), experiment_leaderboard.json,
gemini_wrapper_2026-06-10.json. A design can reuse the entire scoring/loader/stats/gate stack
as-is and only needs to ADD committed golden fixtures (labels + a frozen predictions/feature
substrate) plus a thin file-fed runner.",
    "keyFindings": [
      "(1) WHAT EXISTS / HOW RUN: Two CLI entrypoints at repo root. run_eval.py (->
backend.eval.harness.evaluate) scores DB-stored predictions vs a GT CSV and prints/JSON-dumps
P/R/F1 per stage; read-only, no gate. run_regression.py (->
backend.eval.regression.regress_with_provider) scores DB predictions vs GT from an annotation
provider tier and compares to a snapshotted baseline JSON, returning exit 0=ok / 1=regressed /
3=no-baseline-to-compare. Both read predictions from SQLite (harness.get_predictions:
'candidates'->db.query_candidate_segments, 'final'->db.query_play_segments).
backend.eval.experiment.compare/evaluate_variant is the A/B engine;
```

backend.eval.ablation.run_ablation is the LOSO/LOGO driver (CLI: python -m backend.eval.ablation --features-dir output). fusion_golden, export_wasb_segments, served_gate_a, calibrate_wasb, distill_local are all `python -m backend.eval.<mod>` CLIs. CI (.github/workflows/ci.yml) runs ONLY ruff + mypy + import-smoke + the full pytest suite (--cov-fail-under=70); it does NOT invoke run_regression/run_eval as a gate. So regression today is pytest-only (test_eval_regression.py) using inline tmp_path CSVs + in-memory Database, NEVER a committed golden run.",
      "(2) ALREADY VIDEO-FREE? Partially, and in two senses. The SCORING math (metrics.py, segmentation_metrics.py) and the GT loader (gt_loader.py) are pure stdlib, no video, no GPU, no numpy. harness/regression are video-free in that they read predictions from the DB (no re-detect) ? but the DB itself can only be populated by running the GPU pipeline on video, so 'no committed file -> no run'. The CLOSEST existing video-free-from-files flow is ablation.py: load_videos(features_dir) reads every *_golden_features.json (intervals + shuttle/person/audio feature columns + golden gts baked in) and runs the full ablation 100% on CPU with NO DB and NO video. experiment.compare likewise runs purely on VideoCandidates objects (intervals+features+gts). So the ENGINE to do 'video-free regression from structured files' already exists end-to-end ? what is missing is COMMITTED inputs: the *_golden_features.json files are written to gitignored output/ and are not in the repo.",
      "(3) GOLDEN FIXTURES: eval_baselines/ IS tracked (committed) ? NOT gitignored. It holds heuristic_lovo_n6.json (mean_f1 0.480 + per-video F1 floor for n=6 corpus), experiment_leaderboard.json (append-only A/B records with honest ?F1 CI + sign test), gemini_wrapper_2026-06-10.json. There is NO committed regression_baseline.json (regression.DEFAULT_BASELINE_PATH='eval_baselines/regression_baseline.json' but the file does not yet exist ? it would be committed there once created via --update-baseline). There are NO committed golden label CSVs (.rallies.csv), NO committed WASB trajectory CSVs, and NO committed *_golden_features.json. The golden corpus lives entirely OUTSIDE the repo: output/human_lovo_manifest.json (gitignored) lists 6 videos by name with absolute Windows paths like C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/*.csv and .../Golden Labelled/*.rallies.csv. Tests that need the golden corpus (test_served_gate_a_regression.py) skipif the manifest/files are absent (CI skips; owner box runs).",
      "(4) ARTIFACT FORMATS ARE ALREADY STANDARDISED for a file-fed design: gt_loader.load_gt_intervals accepts BOTH golden CSV conventions through one reader (header start/end OR start_time/end_time, else positional col 0,1; MM:SS/HH:MM:SS or decimal seconds; strict raises with file:line, lenient logs skips). export_wasb_segments writes the golden/slicer schema CSV (rally_number,start_time,end_time,duration,start_mmss,end_mmss) plus a *_vs_golden comparison CSV (type,ref,start_time,end_time,mmss,status,iou,pred_start,pred_end,start_err_s,end_err_s). fusion_golden writes one JSON per video {name,fps,frame_width,candidates:[{start,end,shuttle{5 cols},person{5 cols},label}],gts:[[s,e]...]} consumed by ablation.load_videos and re-scorable forever with no GPU. The regression baseline JSON schema is {'<video_id>::<stage>': {video_id,stage,precision,recall,f1,iou_threshold,detector,gt_hash}} with incommensurability guards (IoU/detector/gt_hash mismatch -> flagged, not silently compared).",
      "(5) FOR A 'VIDEO-FREE REGRESSION FROM COMMITTED STRUCTURED FILES' DESIGN ? REUSE vs ADD. REUSE (do not reinvent): metrics.score/match_intervals/pr_curve; segmentation_metrics.f1_at_overlaps/mean_average_precision/segment_count_ratio; gt_loader.load_gt_intervals (the one CSV reader); regression.regress/save_baseline/load_baselines/gt_hash + the exit 0/1/3 gate contract in run_regression.py; experiment.VideoCandidates + evaluate_variant + compare + record_experiment + the eval_baselines/ committed-location convention; ablation.load_videos (already reads committed-shaped golden_features JSON, DB-free, GPU-free); eval_stats CI+sign-test, splits.assert_disjoint train/eval guard. ADD: (a) commit a small golden fixture set in-repo (label CSVs in the golden schema + a FROZEN predictions/feature substrate ? either tiny *_golden_features.json or a committed predictions JSON/CSV per video) under a tracked dir (tests/ has no fixtures today; eval_baselines/ or a new tests/golden/ are the natural homes); (b) a thin file-fed predictions loader that feeds metrics/regression WITHOUT the DB (today regression.regress only takes predictions via harness.get_predictions -> DB; you need a variant that reads predictions from a committed file); (c) commit the manifest with repo-relative paths (current output/human_lovo_manifest.json is gitignored + absolute-pathed); (d) optionally wire run_regression into CI (today it is not a CI gate)."
    ],
    "dataStructures": [
      {
        "name": "RegressionResult / regression_baseline.json",
        "where": "backend/eval/regression.py (DEFAULT_BASELINE_PATH=eval_baselines/regression_baseline.json ? file NOT yet committed)",

```
        "shapeOrFormat": "Baseline store JSON: { '<video_id>::<stage>':
{video_id,stage,precision,recall,f1,iou_threshold,detector,gt_hash} }. RegressionResult
dataclass adds regressed:bool, reasons:[str], has_baseline:bool, tolerance,
baseline_precision/recall. gt_hash = sha1 of sorted rounded GT intervals (detects label drift).
Incommensurability guards flag IoU/detector/gt_hash mismatch."
      },
      {
        "name": "Golden / slicer CSV schema",
        "where": "backend/eval/export_wasb_segments.py (SEG_FIELDS); read by
gt_loader.load_gt_intervals and tools.rally_slicer.load_rally_labels",
        "shapeOrFormat": "CSV columns:
rally_number,start_time,end_time,duration,start_mmss,end_mmss. The canonical GT reader
(gt_loader) also accepts the collector export form (start,end), positional headerless, and
MM:SS/HH:MM:SS or decimal-seconds timestamps. Comparison CSV (COMP_FIELDS):
type,ref,start_time,end_time,mmss,status,iou,pred_start,pred_end,start_err_s,end_err_s."
      },
      {
        "name": "*_golden_features.json (replay substrate)",
        "where": "produced by backend/eval/fusion_golden.py (run -> out_dir, default gitignored
output/); consumed by backend/eval/ablation.py load_videos and the experiment engine",
        "shapeOrFormat": "{name, fps, frame_width, candidates:[{start, end,
shuttle:{duration,peak_velocity,mean_velocity,active_ratio,net_crossings}, person:{players_in_co
urt,two_sided_motion,mean_player_motion,in_court_ratio,shuttle_player_colocation},
audio?:{audio_hit_count,audio_hit_rate,audio_hit_strength_mean}, label:int}],
gts:[[start,end],...]}. This is the GPU-free, video-free re-scoring input ? exactly the artifact
a file-driven regression would commit."
      },
      {
        "name": "human_lovo_manifest.json",
        "where": "output/human_lovo_manifest.json ? GITIGNORED (output/ is in .gitignore line
39); read by fusion_golden, served_gate_a, distill_local, calibrate_wasb CLIs",
        "shapeOrFormat": "JSON array of [{name, trajectory:<abs path to WASB CSV>, fps,
frame_width, labels:<abs path to .rallies.csv>}]. Paths are absolute and outside the repo
(C:/Users/avidu/Projects/Annotation Setup/...), so this corpus does NOT survive a clean
checkout. n=6 owned golden videos."
      },
      {
        "name": "VideoCandidates / Variant / experiment record",
        "where": "backend/eval/experiment.py; leaderboard committed at
eval_baselines/experiment_leaderboard.json",
        "shapeOrFormat": "VideoCandidates(name, intervals:[(s,e)], features:{col->[per-candidate
floats]}, gts:[(s,e)] or None). Variant = declarative re-scoring recipe
(min_crossings/min_players gates, feature_names, classifier_model, threshold, merge_gap; control
defaults are no-op). Leaderboard row: {timestamp,iou,label_set_hash,num_videos,variant_a/b:{name
,spec_hash,aggregate},delta,honest_delta_f1:{mean_delta,ci_low,ci_high,sign_test:{wins,losses,p_
value}}}."
      },
      {
        "name": "WASB trajectory CSV",
        "where": "parsed by TrackNetRunner.parse_trajectory_csv; consumed by
calibrate_wasb.make_predict_fn, export_wasb_segments, served_gate_a,
distill_local.build_candidates",
        "shapeOrFormat": "CSV columns: Frame,Visibility,X,Y (one row per frame). The per-video
raw substrate from which candidate windows + shuttle features are derived; lives outside the
repo per the gitignored manifest."
      },
      {
        "name": "split manifest (train/eval guard)",
        "where": "backend/eval/splits.py load_split/assert_disjoint",
        "shapeOrFormat": "JSON list of entries each with match_id (or id fallback) + split in
{'train','eval'}; raises on overlap. Pure-logic guard, not yet wired into
distill_local/run_regression (wiring deferred per its docstring)."
      }
    ],
    "videoDependency": "SCORING and the existing experiment/ablation engines are fully video-
```

free. metrics.py, segmentation_metrics.py, gt_loader.py, splits.py, regression.py (the pure parts) need no video/GPU/numpy. experiment.compare and ablation.run_ablation run end-to-end on CPU from in-memory VideoCandidates / committed-shaped *_golden_features.json ? no video, no DB, no Gemini. The DB-read path (harness.get_predictions, regression.regress_with_provider, run_eval/run_regression) is video-free at run time BUT depends on a SQLite DB that only the GPU pipeline (run on real video) can populate ? so without a committed file it cannot run in CI. The FEATURE-PRODUCING step (fusion_golden person detection over the original MP4) and the served-path windowing (served_gate_a reads cached trajectory CSVs, GPU-free with person cue OFF) need the out-of-repo golden corpus (trajectories + .rallies.csv via the gitignored manifest). Net: the design's goal is achievable with ZERO video by committing a frozen predictions/feature substrate + labels and feeding the already-pure scoring/regression layer.",
    "builtVsPlanned": "BUILT & shipped (with passing pytest): pure metrics + segmentation metrics; canonical GT loader (gt_loader, closes the two-loader fork, ADR-008 prereq); DB-read harness + run_eval CLI; baseline-snapshot regression gate + run_regression CLI with exit 0/1/3 and incommensurability guards; experiment A/B engine (compare/evaluate_variant/compare_unlabeled/record_experiment) with committed leaderboard; LOSO/LOGO ablation driver with honest CI+sign-test verdicts (earns_keep/suggestive/inert/structural_protected) + PDF export; fusion_golden feature extractor (with run-telemetry black box); export_wasb_segments for eyeball verification; served_gate_a production-path litmus; calibrate_wasb + distill_local; splits train/eval guard (pure-logic). Committed fixtures: eval_baselines/{heuristic_lovo_n6.json, experiment_leaderboard.json, gemini_wrapper_2026-06-10.json}. PLANNED / NOT built: a committed regression_baseline.json (default path reserved, file absent); committed golden label CSVs / trajectories / *_golden_features.json (all currently land in gitignored output/ or outside the repo); splits wiring into distill_local/run_regression (explicitly deferred in its docstring); regression as a CI gate (CI runs only lint/mypy/import-smoke/pytest+coverage). No 'video-free regression from committed structured files' runner exists yet ? the engine exists, the committed inputs and the file-fed predictions loader do not.",
    "constraints": [
      "Commercial-clean licensing: every dependency code/data/weights MIT/Apache-2.0/BSD only; paid LLMs are inference/serving/fallback only, never a training data source.",
      "Dev container lacks GPU/torch/cv2/numpy ? the full detect pipeline cannot run here; pure-logic eval (metrics, gt_loader, regression core, experiment, ablation, splits) DOES run here. A committed file-driven regression must stay in the pure-logic lane to be CI-runnable.",
      "output/ is gitignored (.gitignore line 39) and *.db is gitignored (lines 36-38) ? so the runtime DB, the manifest, golden_features JSON, and the default regression_baseline.json location do NOT survive a clean checkout unless explicitly relocated. eval_baselines/ is the established TRACKED home for committed eval artifacts.",
      "Honest small-n contract (owner directive 'NUMBERS INFORM, FOUNDATIONS DECIDE'): never a bare mean ? every signal/regression delta ships a bootstrap CI + exact paired sign test (eval_stats); weak/non-significant cues are retained default-OFF, never deleted on a small-n null; structural guards (Gate A) are never auto-demoted on 'no lift'.",
      "Train/eval separation: split unit = match id (never clip/MD5); splits.assert_disjoint must guard any fixture used both for training a scorer and for regression baselines.",
      "gt_hash / label_set_hash / spec_hash fingerprints must travel with any committed baseline so a comparison against changed labels or a changed recipe is flagged incommensurable rather than silently trusted (the existing regression.py and experiment.py guard contract).",
      "eval must not drift from serve: distill_local.PROPOSAL vs SERVED windowing are single-sourced in backend.eval.windowing; served_gate_a pins production operating point (vel 0.02, merge 2.0, min 2.0). A file-driven golden must record which windowing/operating-point it was frozen at."
    ],
    "relevantPaths": [
      "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/metrics.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/segmentation_metrics.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/gt_loader.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/regression.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/harness.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/experiment.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/ablation.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/fusion_golden.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/export_wasb_segments.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/served_gate_a.py",

```
      "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/distill_local.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/splits.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/run_eval.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/run_regression.py",
      "C:/Users/avidu/Projects/badminton-highlight-
indexer/eval_baselines/heuristic_lovo_n6.json",
      "C:/Users/avidu/Projects/badminton-highlight-
indexer/eval_baselines/experiment_leaderboard.json",
      "C:/Users/avidu/Projects/badminton-highlight-
indexer/eval_baselines/gemini_wrapper_2026-06-10.json",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/output/human_lovo_manifest.json",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_eval_regression.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_gt_loader.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_eval_harness.py",
      "C:/Users/avidu/Projects/badminton-highlight-
indexer/tests/test_fusion_golden_telemetry.py",
      "C:/Users/avidu/Projects/badminton-highlight-
indexer/tests/test_served_gate_a_regression.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_export_wasb_segments.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/.github/workflows/ci.yml",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/.gitignore"
    ],
    "risksGaps": [
      "No committed golden fixture survives a clean checkout for an END-TO-END regression: the
n=6 corpus (trajectories + .rallies.csv) is referenced only by the gitignored
output/human_lovo_manifest.json via absolute, machine-specific Windows paths; the
*_golden_features.json substrate lands in gitignored output/. So a 'video-free regression from
committed files' must FIRST commit a (small) frozen substrate + labels + a repo-relative
manifest. eval_baselines/ is the proven tracked home.",
      "regression.regress can only obtain PREDICTIONS via harness.get_predictions, which queries
the SQLite DB. There is no existing path to feed regression committed predictions from a file ?
a thin 'predictions-from-file' loader (or reusing experiment.VideoCandidates ->
evaluate_variant) must be ADDED so the gate runs with no DB.",
      "The two truth-sources can diverge: harness/regression score DB rows (candidate_segments /
play_segments) while experiment/ablation/served_gate_a score trajectory-derived windows. A file-
driven design should pick ONE substrate and record its provenance (windowing/operating-point,
detector, gt_hash) to avoid an incommensurable committed baseline.",
      "Regression is NOT a CI gate today (ci.yml runs only lint/mypy/import-
smoke/pytest+coverage). If the design intends CI enforcement, that wiring is net-new and must
run in the no-GPU lane.",
      "splits.assert_disjoint exists but is NOT wired into regression/distill_local ? committing
a golden fixture risks silent train/eval leakage unless the guard is connected when the fixture
is introduced.",
      "test_served_gate_a_regression.py and the golden-corpus tests skip cleanly when files are
absent; this means the production-path Gate-A litmus is effectively UNVERIFIED in CI ? a
committed mini-substrate would also let that litmus (or a reduced form) actually run in CI.",
      "A committed golden corpus must respect the licensing + minors-exclusion guardrails; raw
match footage cannot be committed, but derived trajectory CSVs / feature JSON / interval labels
are the safe, video-free artifacts to commit."
    ]
  },
  {
    "area": "Golden-set storage, sharing, and regression model ? design + built state",
    "summary": "Golden ground truth is per-video rally [start,end) interval CSVs, keyed by file
MD5 (video_id), produced by the Roora vendor (schema v8) and ingested via the sibling collector.
The eval harness (metrics/regression/experiment) is structurally video-free already ? it re-
scores predictions stored in SQLite and golden labels read from CSV. CRUX: NO doc proposes
committing structured per-video label or feature artifacts to the repo for video-free
regression. The deliberate design is the OPPOSITE: GOLDEN_DATA_SHARING.md keeps the heavy per-
video artifacts (golden-label CSVs, *_golden_features.json, trajectory CSVs, manifests) OUT of
git ? they live in gitignored output/ and sync between the GPU box and CPU dev sessions via a
PRIVATE OneDrive folder, explicitly because they derive from the proprietary corpus and must
never be committed. What IS tracked in git is only the small SCORE artifacts in eval_baselines/
(aggregate + per-video F1, deltas, honest CI/sign-test, label_set_hash, spec hashes) ? no raw
```

labels or features. So a new proposal to commit structured per-video artifacts for video-free regression would EXTEND a gap, not duplicate anything; it must reconcile with (a) the OneDrive sharing convention and (b) the licensing/privacy guardrail that keeps proprietary-derived per-video data private.",
    "keyFindings": [
      "(1) HOW STORED/SHARED TODAY: Golden labels = one CSV per video, one rally per row, [start,end). Two schema layers: the minimal eval-harness format (EVALUATION.md §2: 'start,end' header optional; decimal seconds OR MM:SS/HH:MM:SS; malformed rows fail loudly) read by backend/eval/annotations.py + gt_loader.load_gt_intervals (the canonical golden reader). The richer VENDOR deliverable (ROORA_LABELING_GUIDELINES.md §1) is a per-rally CSV named exactly <video_id>.csv. Keying is by video_id = file MD5 (CLAUDE.md; EVALUATION.md §3; compute_video_id) ? the vendor is told NOT to trim/re-encode because the pipeline IDs by checksum. The golden-label naming convention in code is X.rallies.csv -> X.MP4 (fusion_golden.py::_derive_video_path).",
      "(1b) LOCATION: live golden CSVs + the GPU-extracted *_golden_features.json + trajectory CSVs live in gitignored output/ (per-machine) and scattered local folders. GOLDEN_DATA_SHARING.md (#175, 2026-06-15) wires GPU box <-> CPU dev sessions via a single PRIVATE OneDrive folder: %USERPROFILE%/OneDrive/rally-annotator/golden-data/ with subdirs features/<corpus-tag>/, trajectories/<corpus-tag>/, manifests/, baselines/. corpus-tag = e.g. 2026-06-15-n6. Raw videos are NEVER put there (kept local to GPU box). Optional ergonomics: RALLY_GOLDEN_DIR env var + scripts/sync_golden.py (in #175, not required).",
      "(2) CRUX ? NO doc proposes committing structured per-video LABEL/FEATURE artifacts to the repo for video-free regression. The codebase ALREADY supports video-free regression a different way: detection runs ONCE per video, all per-cue signals + per-candidate features persist to SQLite candidate_segments.stats (JSON) and to output/*_golden_features.json, then ANY variant re-scores OFFLINE on CPU with no GPU/video (EXPERIMENT_HARNESS.md §2, §5.1; ABLATION_LAYER.md; standing lesson 'persist detection once, re-score forever'). But the persistence target is gitignored output/ + private OneDrive, NOT the repo. GOLDEN_DATA_SHARING.md is explicit: 'The shared folder lives OUTSIDE the repo -> never committed. Keep output/ gitignored... features derive from the Proprietary corpus -> keep the OneDrive folder private.' So committing per-video artifacts to git would be a NEW extension, contradicting the current private/OneDrive sharing posture unless reconciled.",
      "(2b) WHAT IS COMMITTED TO THE REPO TODAY (verified via git ls-files): only eval_baselines/*.json SCORE artifacts ? experiment_leaderboard.json (per-A/B-comparison aggregate P/R/F1/mIoU + honest_delta_f1 CI + sign test + label_set_hash + variant spec_hash + promotion/revert records), heuristic_lovo_n6.json (mean + per_video F1 for the 6-match corpus), gemini_wrapper_2026-06-10.json. These contain SCORES and hashes, NOT raw rally labels or feature values. regression.py's DEFAULT_BASELINE_PATH eval_baselines/regression_baseline.json is referenced but NOT present/tracked. So the repo already commits a fingerprint+score layer (label_set_hash, gt_hash/gt_fingerprint) that ties a baseline to a label set WITHOUT storing the labels ? a natural anchor any new per-video-artifact proposal should build on.",
      "(3) SCHEMA + VERSIONING (Roora): ROORA_LABELING_GUIDELINES.md is at VERSION v8 (2026-06-13), declared the SINGLE canonical copy and the sole label to use going forward; it supersedes Drive copies v3/v5/v6/v7 and an internal 'DRAFT v2' (those numbers never matched). The in-repo file is canonical because the fusion fields originate here; a sibling-collector mirror, if staged, must be regenerated from this file. Columns: rally_number, start_mmss, end_mmss, start_time(sec, auto), end_time(sec, auto), shots_count, ending_reason {winner|forced_error|unforced_error|service_fault|let|other}, last_shot_outcome {in|net|out|n_a}, score_before, score_after, notes. Plus a fusion-added per-video INTAKE MANIFEST (NOT in the per-rally CSV): video_id, fps, court_corners (4 px points), net_line (2 px endpoints), singles_or_doubles. ending_reason is shared across all net-separated sports. Three golden rules: ignore all dead time; include EVERY rally (incl. service faults/lets, EXCEPT warm-up and partial edge rallies); a shot = one contact. Quality bar: boundaries within ±0.3s, frame-grid boundaries auto-FAIL. Tier-B (per-shot timestamps, per-shot type) is appendix-only, do-not-start.",
      "(4) BUILT vs PLANNED vs OWNER-GATED. BUILT: GS-1 FileProvider + AnnotationProvider registry (backend/annotations/, providers file+auto); GS-2 regression.regress()/regress_with_provider + save/load_baseline + run_regression.py CI CLI (exit 0/1/2/3); GS-4 AutomatedProvider oracle scaffold; the full eval engine (metrics.py, calibration.py LOVO, classifier.py pinnable JSON model, training.py label/feature glue); EXPERIMENT_HARNESS P1 experiment.py compare()/compare_unlabeled()/record_experiment (shipped 2026-06-13); ABLATION_LAYER Phase-0 ablation.py (LOSO/LOGO, CI+sign test, litmus test); fusion_golden.py GPU extractor -> *_golden_features.json; served_gate_a.py litmus; collector import-local for 3 sports; Roora v8 guideline. PLANNED/owner-gated: GS-3 HumanVendorProvider (parked on Roora pilot confirming quality); GS-5 trigger wiring (CI gate/scheduled);

```
EXPERIMENT_HARNESS P2 per-cue flags+feature persistence, P3 CI promotion gate;
PER_VIDEO_OUTPUT_LAYOUT (design only, not started); internal gold key still pending. GATE-A:
PROMOTED then REVERTED (#146->#151, eval!=serve) ? now OFF-by-default, opt-in only.",
      "(5) STATED CONSTRAINTS ON SHARING GOLDEN DATA. Licensing/privacy: golden sets built ONLY
from licensed/owned footage (founder's collection + ShuttleSet); NO end-user uploads ever flow
to a vendor; all third-party video egress through ONE provider boundary with DPA/no-
reuse/delete-on-delivery/access-logging (GOLDEN_SET_VENDORING.md §2,§4). Repo: per-video golden
artifacts are proprietary-derived -> kept OUT of git, private OneDrive only, no public share
links (GOLDEN_DATA_SHARING.md Guardrails). Minors: EXCLUDED ENTIRELY from the labeled pool ?
footage with children is not annotated, flagged back; minors-exclusion enforced at the pooling
query (ROORA v8 §8 data-handling; QUALITY_ITERATIONS personalization decisions; CLAUDE.md).
Biometrics/identity: no biometric/identity labeling, players referred to by per-video pseudonyms
only, no cross-video person ID, no gait, no pose/skeleton/keypoint; work-for-hire IP assignment
of labels (ROORA v8 §8). Train/eval leakage: split by match/session never by clip; hold out
whole conditions; train+eval in separate catalogs with HARD ingest/export guardrails still a
TODO/BACKLOG (GOLDEN_SET_PHASE1_VIDEOS §5)."
    ],
    "dataStructures": [
      {
        "name": "Golden rally label CSV (eval-harness minimal format)",
        "where": "docs/EVALUATION.md §2; read by backend/eval/annotations.py +
gt_loader.load_gt_intervals; naming X.rallies.csv (fusion_golden.py)",
        "shapeOrFormat": "One CSV per video. Header 'start,end' optional/auto-detected. One
rally per row: start,end as decimal seconds (102.5) OR MM:SS/HH:MM:SS (00:01:12), forms mixable.
Blank lines and # comments ignored. Malformed rows (bad ts, start>=end) fail loudly. video keyed
by file MD5."
      },
      {
        "name": "Roora vendor per-rally CSV (schema v8)",
        "where": "docs/ROORA_LABELING_GUIDELINES.md §1; ingested via sibling collector python -m
src.cli.curate import-local",
        "shapeOrFormat": "One CSV per video named <video_id>.csv, UTF-8, header required, one
row per rally chronological. Cols: rally_number(int 1-based), start_mmss(m:ss.mmm), end_mmss,
start_time(sec auto), end_time(sec auto), shots_count(int),
ending_reason(winner|forced_error|unforced_error|service_fault|let|other),
last_shot_outcome(in|net|out|n_a), score_before, score_after, notes(required when other). Rules:
end>start; no overlaps end[N]<=start[N+1]; ±0.3s boundary tolerance."
      },
      {
        "name": "Per-video intake manifest (fusion-added)",
        "where": "docs/ROORA_LABELING_GUIDELINES.md §A; template INTAKE_MANIFEST_TEMPLATE.csv
(collector side)",
        "shapeOrFormat": "One row per video, recorded once (NOT in the per-rally CSV):
video_id(str), fps(num), court_corners(4 (x,y) px points clockwise from near-left), net_line(2
(x,y) px endpoints), singles_or_doubles(singles|doubles). Worked ex:
court_corners=[(360,980),(1560,980),(1330,300),(590,300)], net_line=[(470,640),(1450,640)]."
      },
      {
        "name": "*_golden_features.json (replayable offline re-scoring substrate)",
        "where": "written by backend/eval/fusion_golden.py to gitignored
output/<name>_golden_features.json; shared via OneDrive features/<corpus-tag>/; NOT committed to
git",
        "shapeOrFormat": "Per-video JSON record: {name, fps, frame_width, candidates:[{start,
end, shuttle{5 fps/res-invariant feats:
duration,peak_velocity,mean_velocity,active_ratio,net_crossings}, person{5 feats:
players_in_court,two_sided_motion,mean_player_motion,in_court_ratio,shuttle_player_colocation},
label(int via IoU match to GT)}], gts}. Re-scored forever on CPU by fusion_compare/ablation ? no
GPU/video."
      },
      {
        "name": "candidate_segments table (SQLite persistence substrate)",
        "where": "backend/infrastructure/database.py (add_candidate_segment :318,
query_candidate_segments :355); output/<db_name>",
        "shapeOrFormat": "Per-candidate row: video_id, detector, start_time, end_time,
chunk_index, confidence, stats(JSON ? per-cue features), detection_params(JSON), human_label,
```

confirmed_segment_id. The persist-once/re-score-forever substrate; keyed by video_id (file MD5)."
      },
      {
        "name": "eval_baselines/experiment_leaderboard.json (TRACKED in git ? scores only)",
        "where": "eval_baselines/experiment_leaderboard.json; written by experiment.record_experiment",
        "shapeOrFormat": "JSON array of comparison/promotion records: {timestamp, iou, label_set_hash, num_videos, variant_a/b:{name,spec_hash,aggregate:{precision,recall,f1,mean_iou}}, delta:{...}, honest_delta_f1:{mean_delta,ci_low,ci_high,n,ci_excludes_zero,sign_test:{wins,losses,ties,n_effective,p_value}}, optional promotion:{cue,promoted_to,decision,reverted,...}}. Contains SCORES + hashes, NO raw labels/features."
      },
      {
        "name": "regression baseline.json (per-video score snapshot)",
        "where": "backend/eval/regression.py DEFAULT_BASELINE_PATH = eval_baselines/regression_baseline.json (referenced; not currently tracked); eval_baselines/heuristic_lovo_n6.json is a tracked sibling",
        "shapeOrFormat": "save_baseline(video_id, stage, precision, recall, f1, ..., gt_fingerprint). RegressionResult: regressed, reasons, gt_hash, baseline P/R, tolerance, has_baseline. heuristic_lovo_n6.json: {name, description, computed, mean_f1, std, per_video:{<video_name>: f1}}. Per-video keyed by video_id/name; ties a baseline to a label set via gt_fingerprint/label_set_hash WITHOUT storing labels."
      }
    ],
    "videoDependency": "The eval/regression machinery is ALREADY video-free at score time and that is the explicit architecture: detection (GPU/WSL, needs the video) runs ONCE per video; all per-cue signals + per-candidate features persist (SQLite candidate_segments.stats JSON and output/*_golden_features.json); then fusion_compare.py / ablation.py / experiment.compare() / run_regression.py re-score ANY number of variants on pure CPU reading only the small JSON + the golden-label CSV ? no video, no GPU, no API calls, deterministic (EXPERIMENT_HARNESS §2, ABLATION_LAYER, EVALUATION §intro). Golden-label CSVs themselves require no video to consume. What is NOT yet video-free / not in the repo: the per-video artifacts (golden-label CSVs, *_golden_features.json, trajectory CSVs, manifests) are gitignored and shared only via private OneDrive (GOLDEN_DATA_SHARING.md), so a fresh git clone alone CANNOT run a regression ? it has the score baselines (eval_baselines/) and the harness code but not the labels/features. A clone CAN run the deterministic synthetic-row unit tests; the golden-litmus tests (test_ablation.py::test_litmus_reproduces_gate_a, test_fusion_compare.py) skipif the output/*_golden_features.json substrate is absent. So committing structured per-video artifacts to the repo is exactly what would make regression runnable from a clone alone ? the unfilled gap.",
    "builtVsPlanned": "BUILT (shipped): GS-1 AnnotationProvider ABC+registry+FileProvider (backend/annotations/, keys file+auto); GS-2 regression.regress()/regress_with_provider + save/load_baseline + run_regression.py CI CLI (exit 0/1/2/3); GS-4 AutomatedProvider oracle scaffold; eval engine metrics.py/calibration.py(LOVO,cross_validate,improvement_report)/classifier.py(pinnable JSON LogisticRegression)/training.py(label_candidates,build_training_set); EXPERIMENT_HARNESS P1 experiment.py compare()/compare_unlabeled()/record_experiment + JSON leaderboard (2026-06-13); honest-stats helpers C1-C4 (eval_stats.py, score_calibration.py, segmentation_metrics.py ? landed, NOT yet wired into default harness flow); ABLATION_LAYER Phase-0 ablation.py with litmus test; fusion_golden.py GPU extractor -> *_golden_features.json; served_gate_a.py litmus + test; FusionSegmenter (fusion_hybrid) default-OFF; collector import-local for 3 sports; Roora v8 guideline (in-repo canonical). PLANNED (not started): GS-3 HumanVendorProvider (parked until Roora pilot confirms quality); GS-5 trigger wiring; EXPERIMENT_HARNESS P2 per-cue enable flags + feature persistence, P3 CI promotion gate; PER_VIDEO_OUTPUT_LAYOUT (L0-L3, design only); internal gold key; hard train/eval-catalog ingest/export guardrails (BACKLOG); GOLDEN_DATA_SHARING optional RALLY_GOLDEN_DIR + sync_golden.py. OWNER-GATED / NOTABLE: GATE-A net_crossings>=2 was PROMOTED to served default (#146) then REVERTED (#151) as an eval!=serve regression (offline +0.074 on proposal windowing did NOT transfer to coarse served windowing, mean served ?F1 -0.008) ? now OFF-by-default, opt-in (indexing.fusion.min_crossings=2). Person cue ?F1 -0.021 and +audio +0.079 both fail the promotion bar at n=6 (CIs span 0) -> stay default-OFF, retained as feature columns. Whole golden-set/owned-model implementation track is owner-gated; corpus = 6 labelled matches in output/human_lovo_manifest.json.",
    "constraints": [

```
      "Golden data built ONLY from licensed/owned footage (founder collection + ShuttleSet); NO end-user uploads ever flow to a vendor (GOLDEN_SET_VENDORING §2)",
      "Per-video golden artifacts are proprietary-derived -> kept OUT of git; shared only via PRIVATE OneDrive (no public links); output/ stays gitignored; do not symlink the shared folder into a tracked path (GOLDEN_DATA_SHARING Guardrails)",
      "Only eval_baselines/*.json SCORE artifacts (aggregate+per-video F1, deltas, CI/sign-test, label_set_hash, spec hashes) are committed to the repo ? NOT raw labels or feature values",
      "Minors EXCLUDED ENTIRELY from the labeled/training pool; footage with children is not annotated and flagged back; minors-exclusion enforced at the pooling query (ROORA v8 §8; CLAUDE.md GDPR Art.9)",
      "No biometric/identity/gait/pose/skeleton/keypoint labeling; players referenced by per-video pseudonyms only, no cross-video person ID; labels are work-for-hire IP-assigned (ROORA v8 §8)",
      "Third-party video egress through ONE provider boundary with DPA, no-reuse, delete-on-delivery, access logging; NDA before any footage shared (GOLDEN_SET_VENDORING §4; ROORA §8)",
      "video_id = file MD5 is the immutable key ? vendor must NOT trim/re-encode/rename source video or the checksum (hence video_id) changes (ROORA v8 §1; EVALUATION §3)",
      "Train/eval split by match/recording-session never by clip; hold out whole conditions; broadcast minimal in eval; HARD ingest/export guardrails so a video can't land in both catalogs are still a TODO/BACKLOG (GOLDEN_SET_PHASE1_VIDEOS §5)",
      "Honest small-N contract: never a bare mean ? promote a cue only when ?F1 CI excludes 0 AND the sign test agrees; structural guards (Gate-A) exempt from auto-demotion (ABLATION_LAYER; QUALITY_ITERATIONS)",
      "Experiment infra is single-tenant/local-first: a JSON leaderboard the size of baseline.json ? explicitly NOT MLflow/W&B/a service (EXPERIMENT_HARNESS §9)",
      "Roora v8 is the SINGLE canonical guideline (supersedes Drive v3/v5/v6/v7 + internal DRAFT v2); the in-repo file is canonical, any collector mirror regenerated from it (ROORA header)",
      "Fast-tier AutomatedProvider oracle MUST be a different model lineage than production (else shared blind spots -> false green); a fast-tier regression is a smoke alarm that escalates to the human tier (GOLDEN_SET_VENDORING §5; GS-4)"
    ],
    "relevantPaths": [
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_DATA_SHARING.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_IMPLEMENTATION.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_VENDORING.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_PHASE1_VIDEOS.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\EVALUATION.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\EXPERIMENT_HARNESS.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\ABLATION_LAYER.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\ROORA_LABELING_GUIDELINES.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\QUALITY_ITERATIONS.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\PER_VIDEO_OUTPUT_LAYOUT.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\eval_baselines\\experiment_leaderboard.json",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\eval_baselines\\heuristic_lovo_n6.json",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\eval_baselines\\gemini_wrapper_2026-06-10.json",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\fusion_golden.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\regression.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\experiment.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\ablation.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\gt_loader.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\annotations.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\annotations\\base.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\infrastructure\\database.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\run_regression.py"
    ],
    "risksGaps": [
```

```
      "CRUX RESTATED for the caller: do NOT assume an existing 'commit per-video artifacts to
repo' proposal exists ? there is none. The only existing per-video-artifact sharing design is
GOLDEN_DATA_SHARING.md, which deliberately keeps them OUT of git (private OneDrive). A new
proposal would EXTEND this gap but must not CONTRADICT: (a) the 'never commit proprietary-
derived data' guardrail, (b) the OneDrive sync convention, (c) the minors/biometric exclusions.
Likely reconciliation: commit only DERIVED, de-identified, label-free or licensing-clean
artifacts (e.g. the *_golden_features.json feature vectors are scale/translation-invariant
counts/ratios, not raw frames or coords ? these MAY be repo-safe where raw CSVs/videos are not;
confirm with owner).",
      "The repo already commits a fingerprint layer (label_set_hash in
experiment_leaderboard.json; gt_fingerprint/gt_hash in regression.py) that binds a score
baseline to a label set without storing labels. Any new per-video-artifact-in-repo design should
build on these hashes for integrity/keying rather than inventing a parallel scheme.",
      "regression.py DEFAULT_BASELINE_PATH points at eval_baselines/regression_baseline.json but
that file is NOT present/tracked ? run_regression.py would hit exit-3 (no-baseline) on a fresh
clone. The tracked baselines are experiment_leaderboard.json + heuristic_lovo_n6.json +
gemini_wrapper_*.json (different shape). A clone cannot currently run the per-video regression
gate end-to-end.",
      "Golden-litmus tests skipif the output/*_golden_features.json substrate is absent ? so on
CI / a fresh clone the golden-litmus coverage is silently skipped; only synthetic-row unit tests
run. Committing the (de-identified) feature substrate would also light up these tests in CI ? a
concrete motivation for the proposal.",
      "Two schema layers coexist (minimal 'start,end' eval CSV vs the rich Roora
<video_id>.csv). The collector ingests the rich one; the indexer harness reads the minimal one
via gt_loader. Any artifact-in-repo proposal must be explicit about which schema it freezes and
how the rich->minimal projection is captured.",
      "Train/eval leakage hard-guardrails (separate catalogs enforced at ingest/export) are
still only a convention + BACKLOG TODO ? a video could currently land in both train and eval by
mistake; relevant if committed artifacts blur the train/eval boundary.",
      "Gate-A eval!=serve revert is the cautionary tale baked into the leaderboard: a number
measured on the harness windowing did not transfer to the served windowing. Any regression model
committed to the repo must record WHICH windowing/operating-point the baseline was measured at,
or it will mispredict served behaviour."
    ]
  },
  {
    "area": "Pipeline data contracts and video-free seam(s) for regression",
    "summary": "The pipeline is a clean 4-stage chain: (1) DETECTION ? the WSL shuttle detector
(WASB/TrackNet) reads frames and emits a per-frame trajectory CSV (`Frame,Visibility,X,Y`); (2)
WINDOWING ? `TrackNetRunner.trajectory_to_action_windows` (the model-agnostic \"brain\") turns
the trajectory into candidate rally window dicts, persisted to `candidate_segments`; (3)
ADJUDICATION/HANDOFF ? fusion gates + AI provider confirm windows into `play_segments`; (4)
STITCH ? `VideoCompiler.compile_highlights` re-encodes/concats the selected `(start,end)` time-
pairs into the reel. Video frames are required in exactly TWO places: the WASB/TrackNet
trajectory pass (Stage 1) and the person-detector pass (fusion P2, default-OFF) ? both consume
the original video by absolute frame index. EVERYTHING from the trajectory CSV onward
(windowing, net-crossing Gate A, person features given boxes, candidate persistence, IoU
scoring, segment selection, merge) is PURE deterministic transformation. The provider handoff
(Gemini) and the final ffmpeg stitch are the only other steps that touch the media file. The
single best video-free regression seam is the TRAJECTORY CSV: freeze `<key>_wasb.csv` per video
and the windowing?gating?scoring chain replays with no GPU/WSL/frames. A second, complementary
seam is the persisted `candidate_segments` rows (post-windowing), and for fusion the per-frame
person-detection dict.",
    "keyFindings": [
      "STAGE SEQUENCE (runtime, trajectory hybrids = the production shuttle path): config load
-> video_id=MD5(file) (backend/utils/hashing.py::compute_video_id, 64KB chunks) -> validators ->
db.add_video (UPSERT) -> segmenter.process_video. Inside
TrajectoryHybridSegmenter.process_video: P1 healthcheck gate -> P2
runner.run_predict(video,out_dir='output/<family>',video_id) writes trajectory CSV;
parse_trajectory_csv -> List[TrajectoryPoint]; trajectory_to_action_windows -> candidate window
dicts; _enrich_candidate_stats per window -> window_stats; db.add_candidate_segment persists
each (full superset) -> P3 _select_for_handoff picks indices; for each,
extract_video_chunk(reencode=True) padded clip -> provider.analyze_video(clip,prompt) -> AI
segment dicts (or skip_ai => windows emitted verbatim) -> P4 db.add_play_segment +
link_candidate_to_segment. Then COMPILE (separate call): db.query_play_segments -> filter by
```

```
stitching.min_shot_count -> time_pairs=[(start_time,end_time)] ->
VideoCompiler.compile_highlights(filepath,time_pairs,out_name).",
      "ARTIFACT 1 ? Trajectory CSV (Stage-1 output, the natural freeze seam): file
`output/<family>/<stem>__<video_id[:12]>_wasb.csv` (WASB) or `<key>_predict.csv` (TrackNet).
Exact columns: `Frame,Visibility,X,Y` (header + order fixed; Visibility==1 means visible; X,Y
are ORIGINAL-frame pixel coords). Parsed to TrajectoryPoint{frame:int, visible:bool, x:float,
y:float}, frame-sorted. This is the complete handoff from the GPU/WSL world to the pure-Python
world.",
      "ARTIFACT 2 ? Candidate window dict (output of trajectory_to_action_windows; in-memory):
{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float,
mean_velocity:float, track_id_count:int (==1 for shuttle trajectory), chunk_index:Optional[int]
(==0 for whole-video trajectory pass)}. Windowing is deterministic: active frame iff visible AND
normalized per-second velocity (dist/frame_width * fps/dframes) > velocity_thresh; active frames
within merge_gap*fps grouped; windows shorter than min_window_duration dropped. fps<=0 ->30.0
and frame_width<=0 ->1280.0 silent fallbacks. visible==False resets prev (occlusion breaks the
track).",
      "ARTIFACT 3 ? candidate_segments DB row (persisted, detector-agnostic fine-tuning table;
THE second freeze seam): columns id, video_id, detector:str, chunk_index, start_time:REAL,
end_time:REAL, duration:REAL(=end-start), confidence:REAL(=min(1.0,peak_velocity)), stats:JSON,
detection_params:JSON, confirmed_segment_id(FK SET NULL), human_label, notes, created_at,
user_id(NULL=local). stats JSON (base trajectory) = {peak_velocity, mean_velocity,
active_frame_count, track_id_count}. detection_params by detector: wasb={detector,velocity_thres
h,merge_gap,min_action_window_duration,weights_path,fusion_substrate?}; tracknet adds
queue_length; yolo={velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector_score_t
hreshold,min_action_window_duration}.",
      "PER-SIGNAL FEATURE COLUMNS ? Gate A (net-crossings):
FusionSegmenter._enrich_candidate_stats ALWAYS adds stats['net_crossings']:float via
rally_gate.gate_windows (pure, from the trajectory; axis/net estimated once per video, cached).
Person cue (default-OFF, only if indexing.fusion.cue_flags.person): adds the 5
PERSON_FEATURE_NAMES = players_in_court, two_sided_motion, mean_player_motion, in_court_ratio,
shuttle_player_colocation (all scale/translation-invariant floats from
fusion_features.compute_person_features). Court mask = optional normalized
indexing.fusion.court_polygon; net=(net_axis,net_position). The learned-classifier shuttle
features (distill_local.FEATURE_NAMES) = duration, peak_velocity, mean_velocity, active_ratio,
net_crossings; the Gen-0 per-frame schema (backend/features/rally_seq.py FEAT_NAMES) = vis_rate,
spd_mean, spd_max, cross_rate, x_std, y_std.",
      "ARTIFACT 4 ? play_segment dict (final segment record, db.add_play_segment /
query_play_segments): {id:int, video_id, start_time:float, end_time:float, duration:float(=end-
start), server, receiver, metadata:dict, events:[...]}. Hybrid P4 metadata = {shot_count:int
(provider shuttle_exchange_count else max(3,int(dur/0.8))), shot_count_confidence:str (provider
value or 'Unknown'/'LocalNoAI'), detector:f'{family}-hybrid-{model_name}'}. Provider (AI)
segment dict consumed by P3 = {start_time, end_time, shuttle_exchange_count?,
shot_count_confidence?} (timestamps offset by clip w_start; _candidate_id stamped internally).",
      "FINAL REEL transform (VideoCompiler.compile_highlights): segments=[(start,end)] ->
_merge_overlapping (parse times, drop end<=start, sort, merge overlapping/abutting ranges so a
rally is never stitched twice) -> per-clip ffmpeg re-encode (libx264/superfast/crf22, identical
settings, optional watermark via -vf) -> `-f concat -c copy` -> output/<name>.mp4. Selection
upstream: API filters by stitching.min_shot_count (segments without shot_count are exempt) and
by req.segment_ids, then time_pairs=[(s['start_time'],s['end_time'])].",
      "TWO ALTERNATE PATHS that bypass the trajectory seam: `gemini` segmenter (pure-AI, no
P2/candidates ? chunks the whole video, uploads, provider returns segments directly to
play_segments) and `motion` (pixel-absdiff baseline with FABRICATED random metadata/events).
These two are the _FINAL_ONLY_SEGMENTERS in eval/batch.py ? they have no candidate stage and
cannot be frozen at the trajectory seam. The video-free seam analysis applies to the trajectory
hybrids (wasb_hybrid/tracknet_hybrid/fusion_hybrid), which are the rally-quality track."
    ],
    "dataStructures": [
      {
        "name": "Trajectory CSV",
        "where": "output/<family>/<stem>__<video_id[:12]>_wasb.csv (WASB) / <key>_predict.csv
(TrackNet); written in WSL, read by
backend/pipeline/detectors/tracknet_runner.py::parse_trajectory_csv",
        "shapeOrFormat": "CSV header `Frame,Visibility,X,Y` (fixed order); rows: Frame:int,
Visibility:int(1=visible), X:float px, Y:float px ->
TrajectoryPoint{frame:int,visible:bool,x:float,y:float} frame-sorted"
```

```
      },
      {
        "name": "Candidate window dict",
        "where": "output of TrackNetRunner.trajectory_to_action_windows (tracknet_runner.py);
in-memory, consumed by trajectory_hybrid P2/P3",
        "shapeOrFormat": "{start:float(sec), end:float(sec), active_frame_count:int,
peak_velocity:float, mean_velocity:float, track_id_count:int(==1),
chunk_index:Optional[int](==0)}"
      },
      {
        "name": "candidate_segments row",
        "where": "backend/infrastructure/database.py (add_candidate_segment /
query_candidate_segments), table candidate_segments",
        "shapeOrFormat": "id, video_id, detector:str, chunk_index, start_time:REAL,
end_time:REAL, duration:REAL, confidence:REAL(min(1.0,peak_velocity)),
stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count[,net_crossings,+5
person feats]}, detection_params:JSON, confirmed_segment_id, human_label, notes, created_at,
user_id"
      },
      {
        "name": "Per-signal feature columns (fusion)",
        "where": "backend/pipeline/detectors/fusion_features.py::PERSON_FEATURE_NAMES +
rally_gate net_crossings; persisted into candidate_segments.stats by
fusion_hybrid.py::_enrich_candidate_stats",
        "shapeOrFormat": "net_crossings:float (Gate A, always); person (flag-gated):
players_in_court, two_sided_motion, mean_player_motion, in_court_ratio,
shuttle_player_colocation (all floats, scale/translation-invariant)"
      },
      {
        "name": "Person per-frame detections",
        "where": "backend/pipeline/detectors/person_detector.py::detections_per_frame (fusion
P2, needs video+GPU)",
        "shapeOrFormat": "{'frames': {frame_idx:int -> [(track_id:int, bbox:(x1,y1,x2,y2)),
...]}, 'fps':float, 'frame_width':float, 'frame_height':float}; frame_idx aligns 1:1 with
trajectory .frame"
      },
      {
        "name": "play_segment dict (final)",
        "where": "backend/infrastructure/database.py::add_play_segment / query_play_segments,
table play_segments",
        "shapeOrFormat": "{id:int, video_id, start_time:float, end_time:float,
duration:float(=end-start), server, receiver,
metadata:dict{shot_count:int,shot_count_confidence:str,detector:'<family>-hybrid-<model>'},
events:[...]}"
      },
      {
        "name": "AI/provider segment dict",
        "where": "provider.analyze_video output, consumed by trajectory_hybrid P3 and
gemini.py",
        "shapeOrFormat": "{start_time:float(decimal sec), end_time:float,
shuttle_exchange_count?:int, shot_count_confidence?:'Low'|'Medium'|'High'}; +_candidate_id
stamped internally; times offset by clip start"
      },
      {
        "name": "Final reel time_pairs",
        "where": "backend/main.py::compile_highlights -> VideoCompiler.compile_highlights
(compiler.py)",
        "shapeOrFormat": "List[Tuple[start:float, end:float]] -> _merge_overlapping (sort+merge)
-> per-clip reencode + concat -c copy -> output/<name>.mp4"
      },
      {
        "name": "GT / golden CSV (eval comparison)",
        "where": "backend/eval/annotations.py::load_annotations (start,end) and
calibrate_wasb.load_golden_gts (rally_number,start_time,end_time)",
        "shapeOrFormat": "generic: `start,end` 2-col (RAISES on bad row); golden/slicer:
```

```
`rally_number,start_time,end_time[,duration,start_mmss,end_mmss]` (SILENTLY skips bad rows)"
      }
    ],
    "videoDependency": "Video/frames are REQUIRED in exactly two compute steps, both at the
front of the trajectory-hybrid pipeline: (A) the shuttle trajectory pass ?
WasbRunner/TrackNetRunner.run_predict stages the video into WSL and runs the GPU CNN over
decoded frames to emit the `Frame,Visibility,X,Y` CSV (backend/pipeline/detectors/wasb_runner.py
+ wasb_infer.py, GPU/WSL/cv2); and (B) the person-detector pass ?
PersonDetector.detections_per_frame seeks the ORIGINAL video by absolute frame range (fusion P2,
default-OFF, torch/cv2/GPU). In addition the media file is touched (not 'frames' analysis but
read+transcode) at the AI handoff (extract_video_chunk re-encodes a padded clip and uploads to
Gemini) and at the final stitch (VideoCompiler re-encodes/concats clips with ffmpeg). EVERYTHING
ELSE is pure deterministic Python on numbers/strings: parse_trajectory_csv,
trajectory_to_action_windows (windowing brain), rally_gate (net-crossings Gate A),
fusion_features (person features given boxes ? pure, no cv2/torch), db.add_candidate_segment /
add_play_segment persistence, eval/metrics IoU scoring, _select_for_handoff gating, and
_merge_overlapping. So once the trajectory CSV (and, for fusion, the per-frame person dict)
exists, no video, GPU, WSL, ffmpeg, or cloud is needed to reproduce windowing, gating, candidate
rows, and IoU scores.",
    "builtVsPlanned": "BUILT and live: full trajectory-hybrid path (wasb_hybrid is the live
shuttle substrate), parse_trajectory_csv + trajectory_to_action_windows (pure, static, model-
agnostic), candidate_segments persistence (schema v5), play_segments, VideoCompiler stitch with
overlap-merge, rally_gate.py (Gate A net-crossings ? PURE, unit-tested), fusion_features.py
(PURE person-feature math, unit-tested), FusionSegmenter (registered fusion_hybrid, default-OFF,
two identity hooks; net_crossings always persisted, person features + served Gate A/B flag-gated
default-0/OFF), eval harness (metrics/calibration/distill_local/rally_seq_proto), the
skip_ai_handoff fully-local mode. The eval drivers ALREADY consume the trajectory CSV directly
and replay windowing/gating/scoring with no video ? i.e. the video-free seam is de-facto
operational in eval/ (calibrate_wasb.make_predict_fn parses the trajectory once and closes over
trajectory_to_action_windows; eval_partial_labels, export_wasb_segments, calibrate_local all do
this). PLANNED/owner-gated: per-video output layout (PER_VIDEO_OUTPUT_LAYOUT.md, NOT started ?
trajectory dir still hardcoded 'output/<family>'), P3 learned fusion (person features into
classifier, gated on golden labels), P4 audio/pose cues, served Gate-A-in-production promotion
(thresholds default 0). NO formal frozen-fixture regression harness keyed on the trajectory CSV
exists as a tracked test yet ? the seam is exploited ad-hoc by eval drivers but not pinned as a
video-free regression suite.",
    "constraints": [
      "Trajectory CSV columns are FIXED: `Frame,Visibility,X,Y` header + order; X,Y are
ORIGINAL-frame pixels (affine-resized back from the 512x288 detector input). A frozen fixture
must capture exactly this.",
      "Windowing needs fps + frame_width alongside the CSV (it normalizes velocity by
frame_width and converts frame->sec by fps). _probe_fps_width reads these from the video via
cv2; for a video-free replay they MUST be captured/pinned (fallbacks 30.0/1280.0 are silent and
would skew results). This is the one non-CSV input the windowing brain needs.",
      "Person features need the per-frame detection dict {frame_idx -> [(track_id,bbox)]} +
frame_width/height; frozen separately from the CSV. Their frame indices must align 1:1 with the
trajectory .frame.",
      "Candidate persistence is detector-agnostic by design (stats/detection_params JSON) ? a
frozen candidate_segments row is a valid second seam but is one transform DOWNSTREAM of the CSV
(windowing already applied), so it cannot regress the windowing brain itself.",
      "gemini and motion segmenters have NO candidate/trajectory stage (final-only) ? they
cannot use the trajectory seam; their regression must freeze provider responses or accept they
touch video.",
      "eval/metrics.match_intervals is GREEDY (not Hungarian) and deterministic; empty matches
yield 0.0 not NaN ? stable for golden assertions.",
      "License guardrail: any frozen-fixture tooling must stay MIT/Apache/BSD; WASB weights are
academic-data-trained (confirm terms before commercial deploy)."
    ],
    "relevantPaths": [
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\tracknet_runner.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\wasb_runner.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\base.py",
```

```
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\rally_gate.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\fusion_features.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\person_detector.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\trajectory_hybrid.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\fusion_hybrid.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\base.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\infrastructure\\database.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\stitcher\\compiler.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\utils\\hashing.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\metrics.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\calibrate_wasb.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\harness.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\features\\rally_seq.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\main.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\CODE_MAP.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\HOW_RALLY_DETECTION_WORKS.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\MULTI_SIGNAL_FUSION_PLAN.md"
    ],
    "risksGaps": [
      "SOURCES OF NON-DETERMINISM downstream of the trajectory-CSV seam ? these must be
controlled or excluded from frozen assertions: (1) FLOAT ? velocity =
(dist/frame_width)*(fps/dframes), median/spread in rally_gate, normalized person motion: all
float ops, platform-stable but assert with tolerance not exact equality; (2) ORDERING/THREADING
? P3 uses ThreadPoolExecutor(max_workers=ai_max_workers); ai_segments.extend(future.result())
iterates futures in submission order so it IS deterministic, but the underlying provider calls
are not (network) ? freeze at the windowing/candidate seam BEFORE P3, or stub the provider; (3)
TIMESTAMPS ? candidate_segments/play_segments rows carry created_at DEFAULT CURRENT_TIMESTAMP
and the DB autoincrement id ? exclude these columns from any golden comparison; (4) DB id NON-
DETERMINISM ? play_segment.id is AUTOINCREMENT, link_candidate_to_segment writes it ? compare on
(start_time,end_time,stats) not id; (5) ffmpeg/codec ? the final reel bytes are NOT reproducible
across ffmpeg builds (libx264 version, NVENC vs libx264 fallback, watermark font) ? never assert
reel bytes; assert the time_pairs + merge output instead.",
      "fps/frame_width are probed from the VIDEO (cv2) inside the segmenter, not stored in the
CSV ? a video-free replay that forgets to pin them silently falls back to 30.0/1280.0 and
produces different windows. The frozen fixture MUST include the (fps, frame_width) used.",
      "No tracked frozen-fixture regression test currently pins the trajectory-CSV -> windows ->
candidates -> IoU chain end-to-end as a video-free suite; the seam is only exercised ad-hoc by
eval CLIs. This is the gap a regression harness would fill.",
      "trajectory_to_action_windows treats visible==False as a hard track break (occlusion gap
resets prev) and silently clamps fps<=0->30 / width<=0->1280 ? a fixture with degenerate values
would mask real regressions.",
      "Two segmenters (gemini, motion) bypass the seam entirely (no candidate stage); motion
additionally fabricates RANDOM metadata/events (random.sample/uniform) so it is inherently non-
deterministic and unsuitable for golden assertions.",
      "The person-feature path (fusion, default-OFF) needs a SEPARATELY frozen per-frame
detection dict; cv2 frame-seek accuracy varies by codec (±a few frames on real footage), so the
captured dict must come from the same decode the fixture pins."
    ]
  },
  {
    "area": "Test suite execution model, committed fixture data, mocking conventions, and the
right home for new auto-discovered per-video regression fixtures",
    "summary": "Tests are pure-pytest auto-discovery (no test runner framework wrapping):
```

`pytest.ini` sets `testpaths = tests`, `run_all_tests.py` is just a thin `subprocess` wrapper around `python -m pytest` (defaults to `-q`, passes all args through, prints a PASS/FAIL banner, returns pytest's exit code). There is NO conftest.py and NO committed CSV/JSON test fixtures under `tests/` or `backend/eval/` ? every test synthesizes its fixtures inline into pytest's `tmp_path` (CSVs via `csv.writer`/`p.write_text`, manifests + golden-feature JSON via `json.dump`). The only committed data files are 3 small JSONs in `eval_baselines/` (the no-regression baselines/leaderboard, tracked deliberately) plus `config.json` and `artifacts/rally_tal_gen0/0.1.0/{manifest,weights}.json`. The real per-video golden corpus (trajectory CSVs, *.rallies.csv labels, *_golden_features.json, human_lovo_manifest.json) lives ONLY in the gitignored `output/` dir on the owner's box ? it is NOT committed, so any test depending on it (test_served_gate_a_regression.py) skips cleanly when absent. There IS a clean load?transform?assert pattern to follow (fusion_compare.load_videos reads *_golden_features.json), and an existing manifest-driven, skip-if-absent per-video regression test to mirror (test_served_gate_a_regression.py). New committed, auto-discovered per-video fixtures + a parametrized test should slot into a NEW fixtures dir UNDER tests/ (e.g. tests/fixtures/golden/) with the parametrized test in tests/test_*.py globbing that dir.",
    "keyFindings": [
      "DISCOVERY/RUN: pytest.ini (repo root) is the only pytest config ? a single line `testpaths = tests`. No pyproject [tool.pytest], no markers registered, no conftest.py anywhere in the repo. run_all_tests.py (repo root, 60 lines) defaults to `-q`, passes every CLI arg straight through to `python -m pytest`, and propagates pytest's exit code (CI/pre-PR friendly). The full-suite lane enforces a 70% coverage floor (CODE_MAP §0); run with `python run_all_tests.py --cov=backend --cov-report=term-missing`.",
      "SKIP/OPTIONAL-DEP CONVENTIONS: two idioms in use ? (a) `pytest.importorskip('cv2')` / `importorskip('obsreport')` at module top to skip whole modules when a heavy/optional dep is absent (CI lacks GPU/torch/cv2); (b) a module-level `_skip = pytest.mark.skipif(not _SPECS, reason=...)` decorator gated on whether real golden artifacts exist on disk. These are how real-video/GPU tests degrade to SKIP rather than ERROR in CI.",
      "NO COMMITTED TEST FIXTURES: `Glob tests/**/*.csv`, `tests/**/*.json`, `backend/eval/**/*.csv|json` all return NOTHING. Every fixture is built inline into `tmp_path`. CSV GT is written with `csv.writer` or `p.write_text('start,end\\n0,10\\n...')` (test_gt_loader.py, test_eval_regression.py). Per-video golden-feature JSON is written with `json.dump({name, fps, frame_width, candidates:[{start,end,shuttle{},person{},label}], gts:[[a,b]]}, ...)` (test_fusion_compare.py::_write_video). Manifests are `json.dump([{name, trajectory, fps, frame_width, labels}], ...)` (test_fusion_golden_telemetry.py::_write_manifest).",
      "COMMITTED DATA (the only tracked non-code data): eval_baselines/heuristic_lovo_n6.json (689 B, 15 lines ? the LOVO floor + per_video f1 map), eval_baselines/experiment_leaderboard.json (4.7 KB, 139 lines ? A/B variant rows with honest_delta_f1 CIs), eval_baselines/gemini_wrapper_2026-06-10.json (1.98 KB). Plus config.json (2.7 KB) and artifacts/rally_tal_gen0/0.1.0/{manifest,weights}.json (untracked here, ADR-008: manifest committed / weights gitignored). eval_baselines/ is deliberately tracked so a clean checkout still has the no-regression baselines.",
      "MOCKING ? DETECTOR: injected, not patched. extract_video(spec, detector=None,...) takes an optional `detector` with a `detections_over_windows(video, windows, frame_skip, progress_cb)` duck-typed interface; tests pass a `_StubDetector` returning canned `{frames, fps, frame_width, frame_height}` dicts (test_fusion_compare.py). fusion_golden.run() is tested by monkeypatching `fusion_golden.extract_video` wholesale.",
      "MOCKING ? WSL: patch the single seam method `_wsl_bash` on a WasbRunner instance with `unittest.mock.patch.object(r, '_wsl_bash', side_effect=[_proc(rc,stdout,stderr), ...])`, where `_proc` is a `types.SimpleNamespace(returncode, stdout, stderr)` stand-in for a CompletedProcess. Higher-level seams `_stage_video`, `_stage_infer_script`, `_wsl_rm_rf`, `run_predict` are also patched. No real `wsl.exe` is ever invoked (test_wasb_runner.py).",
      "MOCKING ? VIDEO/GPU: no real MP4 decode in the suite. fusion_golden derives the MP4 path from the labels filename (X.rallies.csv -> X.MP4) but the stubbed detector never opens it. TrackNetRunner.parse_trajectory_csv (Frame,Visibility,X,Y DictReader) is monkeypatched to return canned TrajectoryPoint lists, OR fed a tiny real CSV string. RunTelemetry (nvidia-smi/psutil) is made hermetic by monkeypatching gpu_name + injecting deterministic gpu_fn/sys_fn/clock collectors (test_fusion_golden_telemetry.py).",
      "EXISTING load?transform?assert PATTERN TO FOLLOW: backend/eval/fusion_compare.py::load_videos(features_dir) globs `*_golden_features.json`, loads each into experiment.VideoCandidates, and compare() runs honest LOVO; tests assert on the resulting ?F1/CI/sign-test (test_fusion_compare.py). This is the canonical 'load artifact -> run transform -> assert' shape, but today the artifacts are written to tmp_path inside the test, never committed.",

```
      "EXISTING PER-VIDEO REGRESSION TEST TO MIRROR: tests/test_served_gate_a_regression.py is a
manifest-driven, per-video, skip-if-absent regression litmus. It resolves MANIFEST from
`<repo_root>/output/human_lovo_manifest.json` (env-overridable via SERVED_GATE_A_MANIFEST),
`_golden_specs()` keeps only specs whose trajectory+labels files exist, and `_skip = skipif(not
_SPECS, ...)` skips the whole module in CI where the gitignored corpus is absent. It loops over
each golden video and asserts served-path F1/over-seg invariants. This is the closest template
for new per-video regression fixtures ? the gap a committed-fixture approach would close is that
it currently CANNOT run in CI (corpus uncommitted).",
      "GT LOADER CONTRACT (load-bearing for fixtures):
backend/eval/gt_loader.py::load_gt_intervals reads BOTH golden-CSV conventions ? header
`start,end` (collector curate export) OR `start_time,end_time` (rally-annotator .rallies.csv,
extra cols ignored), else positional cols 0,1; timestamps accept decimal seconds or
MM:SS/HH:MM:SS; strict=True raises with file:line, strict=False skips+logs. Any committed CSV
fixture must conform to one of these two shapes."
    ],
    "dataStructures": [
      {
        "name": "Per-video golden-feature JSON (*_golden_features.json)",
        "where": "Written by backend/eval/fusion_golden.py::extract_video/run into gitignored
output/; consumed by backend/eval/fusion_compare.py::load_videos; synthesized inline in
tests/test_fusion_compare.py::_write_video",
        "shapeOrFormat": "{\"name\": str, \"fps\": float, \"frame_width\": float,
\"candidates\": [{\"start\": float, \"end\": float, \"shuttle\": {5 SHUTTLE_FEATURE_NAMES:
float}, \"person\": {5 PERSON_FEATURE_NAMES: float}, \"label\": int(0|1)}], \"gts\":
[[start,end], ...]}. Tiny (a handful of candidates per video in synthetic tests). This is the
natural unit for a committed per-video regression fixture."
      },
      {
        "name": "LOVO manifest (human_lovo_manifest.json)",
        "where": "Lives only in gitignored output/; consumed by fusion_golden.run,
served_gate_a, calibrate_local; synthesized inline in
tests/test_fusion_golden_telemetry.py::_write_manifest",
        "shapeOrFormat": "JSON array of specs: [{\"name\": str, \"trajectory\": \"<name>.csv\",
\"fps\": float, \"frame_width\": float, \"labels\": \"<name>.rallies.csv\", optional
\"video_path\": str}]"
      },
      {
        "name": "Trajectory CSV (TrackNet/WASB predict output)",
        "where": "backend/pipeline/detectors/tracknet_runner.py::parse_trajectory_csv
(DictReader); referenced by manifest 'trajectory' field; only exists in gitignored output/",
        "shapeOrFormat": "CSV header `Frame,Visibility,X,Y`; one row per frame; Visibility==1
means shuttle visible. Parsed into TrajectoryPoint(frame:int, visible:bool, x:float, y:float)."
      },
      {
        "name": "Golden GT / rally-label CSV (two conventions)",
        "where": "backend/eval/gt_loader.py::load_gt_intervals (canonical),
annotations.load_annotations, calibrate_wasb.load_golden_gts; synthesized inline in
test_gt_loader.py / test_eval_regression.py",
        "shapeOrFormat": "Either header `start,end` (collector export) OR
`rally_number,start_time,end_time,ending_reason,sport` (rally-annotator .rallies.csv), OR
headerless positional cols 0,1. Timestamps: decimal seconds or MM:SS/HH:MM:SS. ->
List[(start:float, end:float)]"
      },
      {
        "name": "Eval baseline / leaderboard JSON",
        "where": "Committed in eval_baselines/; written by
backend/eval/experiment.py::record_experiment + regression.save_baseline; read by
run_regression.py no-regression gate",
        "shapeOrFormat": "heuristic_lovo_n6.json: {name, description, computed, mean_f1, std,
per_video:{vid:f1}}. experiment_leaderboard.json: array of {timestamp, iou, label_set_hash,
num_videos, variant_a/b:{name,spec_hash,aggregate:{precision,recall,f1,mean_iou}}, delta,
honest_delta_f1:{mean_delta,ci_low,ci_high,n,ci_excludes_zero,sign_test}}"
      },
      {
        "name": "WSL CompletedProcess stand-in (_proc)",
```

```
        "where": "tests/test_wasb_runner.py",
        "shapeOrFormat": "types.SimpleNamespace(returncode:int, stdout:str, stderr:str) ? fed
via patch.object(runner, '_wsl_bash', side_effect=[...]) to mock every WSL bash call"
      }
    ],
    "videoDependency": "No test in the suite decodes a real video. The pipeline cannot run here
(dev container lacks GPU/torch/cv2/numpy per CLAUDE.md). Real-video/GPU paths are handled three
ways: (1) inject a stub detector with a duck-typed detections_over_windows() so no MP4 is
opened; (2) monkeypatch TrackNetRunner.parse_trajectory_csv / fusion_golden.extract_video to
return canned data; (3) for tests that genuinely need the real golden corpus (trajectory CSVs +
.rallies.csv labels + MP4s), gate the whole module behind pytest.importorskip('cv2') and/or a
skipif on whether the gitignored output/human_lovo_manifest.json and its referenced files exist
on disk ? these SKIP in CI and only run on the owner's box. The real 6-video golden corpus
(adarsh_avi_singles, kushagra_singles, largetest_doubles, gbaaddy,
testlarge_short/largetest_short, mahadevpura_singles) is NOT committed; only their derived
*_golden_features.json sit in gitignored output/.",
    "builtVsPlanned": "BUILT: full pytest auto-discovery suite (~85 test_*.py files), the
run_all_tests.py wrapper, the load?transform?assert pattern (fusion_compare.load_videos), the
inline-tmp_path fixture idiom, the skip-if-corpus-absent per-video regression litmus
(test_served_gate_a_regression.py), and the committed eval_baselines/ no-regression baselines.
NOT BUILT / GAP the task targets: there are currently NO committed per-video regression fixtures
and NO parametrized test that auto-discovers committed per-video artifacts ? the existing per-
video regression test depends on the uncommitted gitignored output/ corpus, so it never runs in
CI. A new tests/fixtures/golden/ dir of small committed *_golden_features.json + a parametrized
test would make per-video regression CI-runnable for the first time. (No conftest.py exists yet
either, so shared fixture plumbing would be a new addition.)",
    "constraints": [
      "Commercial-clean licensing + privacy: committed fixtures must EXCLUDE minors from any
training/eval pool and must not embed paid-LLM (Gemini) outputs as a training data source
(CLAUDE.md guardrails). Real-player footage frames cannot be committed; synthetic or trajectory-
only/feature-only JSON is the safe form.",
      "Fixtures must be TINY and offline: the suite is fully mocked, no GPU/WSL/network, and CI
has a 70% coverage floor. Committed *_golden_features.json should be small (a handful of
candidates), like the synthetic ones in test_fusion_compare.py ? not the full real corpus.",
      "CSV fixtures MUST conform to one of the two gt_loader conventions (header start,end |
start_time,end_time, or positional) or they will be rejected/skipped by load_gt_intervals
(strict mode raises with file:line).",
      "File-placement rule (CODE_MAP §0): tests go in tests/test_*.py; eval baselines go in
tracked eval_baselines/; pipeline run outputs go in gitignored output/. There is no existing
tests/fixtures or tests/data dir ? adding one is a new convention but consistent with 'a test ->
tests/'. Do NOT put fixtures in output/ (gitignored, won't survive checkout/CI).",
      "Branch from origin/master, stage explicit paths (no git add -A), re-verify HEAD before
commit ? shared checkout / concurrent sessions (CLAUDE.md + memory gotcha-shared-checkout-
branch-collisions).",
      "MANIFEST.in / hatchling packaging: pyproject lists packages=[backend, training]; a
tests/fixtures dir is fine for pytest (run from source tree) but would not be packaged into the
wheel ? acceptable since tests aren't shipped."
    ],
    "relevantPaths": [
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\run_all_tests.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\pytest.ini",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\pyproject.toml",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.gitignore",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_eval_regression.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_gt_loader.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\tests\\test_fusion_golden_telemetry.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_fusion_compare.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\tests\\test_served_gate_a_regression.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\tests\\test_per_user_regression.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_wasb_runner.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\fusion_golden.py",
```

```
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\fusion_compare.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\gt_loader.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\regression.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\per_user_regression.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\tracknet_runner.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\eval_baselines\\heuristic_lovo_n6.json",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\eval_baselines\\experiment_leaderboard.json",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\CODE_MAP.md"
    ],
    "risksGaps": [
      "RECOMMENDED PLACEMENT for new auto-discovered per-video fixtures: create
tests/fixtures/golden/ holding small committed *_golden_features.json (one per synthetic video,
matching the fusion_golden output schema), and a new tests/test_golden_regression.py that does
`FIXTURE_DIR = Path(__file__).parent/'fixtures'/'golden'` then
`@pytest.mark.parametrize('fixture', sorted(FIXTURE_DIR.glob('*_golden_features.json')),
ids=lambda p: p.stem)`. This mirrors fusion_compare.load_videos (glob+json.load) but reads
COMMITTED files instead of tmp_path, and mirrors test_served_gate_a_regression.py's per-video
loop ? except it runs in CI because the fixtures are committed. Resolve paths via
Path(__file__).parent, NEVER cwd (agent cwd resets between bash calls; tests may run from any
dir).",
      "There is no conftest.py today. If the parametrized test needs a shared loader/fixture,
add tests/conftest.py (new file) with a small `load_golden_fixture` helper or a `golden_videos`
fixture ? but a module-local glob is simpler and matches the existing zero-conftest convention;
prefer that unless multiple test files need it.",
      "Do NOT reuse output/ for committed fixtures ? it is gitignored and won't survive a clean
checkout or exist in CI (that is exactly why test_served_gate_a_regression.py skips in CI). The
fixtures must be inside tests/ to be tracked and discovered.",
      "Privacy/licensing risk if real footage-derived data is committed: the real 6-video corpus
is owner footage; commit only SYNTHETIC or fully-anonymized feature JSON (no frames, no minors),
consistent with the inline-synthetic data already used in test_fusion_compare.py. If anchoring
on real numbers, store only the derived F1/baseline in eval_baselines/ (already the convention),
not raw player data.",
      "A committed-fixture regression test should assert against a committed expected value
(e.g. an eval_baselines/ entry or an inline EXPECTED constant) using the existing
regression.regress / experiment.compare / metrics.score transforms ? reuse those, do not write
new scoring math (CODE_MAP warns calibrate_wasb.load_golden_gts + experiment harness are widely-
shared APIs).",
      "Marker hygiene: no custom pytest markers are registered (no [tool.pytest] markers
config). If the new test introduces a marker (e.g. @pytest.mark.golden), register it in
pytest.ini's `markers =` or it will emit PytestUnknownMarkWarning; simpler to rely on
parametrize + skipif like the rest of the suite."
    ]
  },
  {
    "area": "Repo constraints for committing structured regression fixtures (file-placement,
gitignore, git-lfs, schema/version conventions, licensing/privacy guardrails, size budget)",
    "summary": "The repo deliberately commits NO raw/derived video data today: there are zero
tracked .csv/.jsonl/fixture files, tests/ has no data dirs (tests build synthetic data in-code),
and the ONLY committed structured data is tiny aggregate-metric JSON in eval_baselines/ (e.g.
heuristic_lovo_n6.json ? mean_f1 + per-video scores, no per-frame data). The file-placement
table (CODE_MAP §0) routes pipeline outputs (trajectories, features, DBs, frames) to gitignored
output/, and a dedicated doc ? docs/GOLDEN_DATA_SHARING.md (#175) ? explicitly states the golden
corpus is PROPRIETARY and its derived features must live in a PRIVATE OneDrive folder OUTSIDE
the repo, never committed, with a guardrail forbidding symlinking it into a tracked path. git-
lfs is INSTALLED (3.5.1, global filters) but NOT in use (no .gitattributes anywhere). On the
licensing/privacy question: committing detector-derived numeric streams (shuttle X/Y, segment
intervals, person/audio features) is defensible because the golden videos are owned/licensed by
the founder (self-recorded + ShuttleSet, never end-user uploads) and the streams are abstract
numbers, not frames ? BUT the repo's OWN convention currently says keep them out of git (private
OneDrive), and minors must be excluded, so any decision to commit even derived data is a
```

product/privacy call that is owner-gated, not a free engineering choice.",
    "keyFindings": [
      "(1) WHERE FIXTURES BELONG: Per docs/CODE_MAP.md §0 file-placement table ? tests go in tests/test_*.py (pytest auto-discover, 70% coverage floor, must be pure/offline/mocked, no GPU/WSL/network). Small TRACKED metric baselines go in eval_baselines/ ('Tracked ? survives a clean checkout; the no-regression gate reads it'). Model-artifact MANIFESTS go in artifacts/<name>/<ver>/ (manifest committed, weights gitignored ? ADR-008). Pipeline run outputs (reels, TRAJECTORIES, DBs, frames) go in output/ = GITIGNORED.",
      "(2) WHAT IS GITIGNORED: .gitignore ignores output/, output_dir/, scratch/, .venv/, all *.db/*.db-journal/*.db-wal, *.pt (weights), .env*, coverage, __pycache__, and specific backend/static highlight mp4s. NOTE: artifacts/ itself is NOT in .gitignore (it currently shows as untracked in git status) ? only model WEIGHTS inside it are meant to be gitignored per ADR-008, the manifest is committed. scratch/ is fully gitignored+excluded from ruff/pytest.",
      "(3) GIT-LFS: git-lfs v3.5.1 is installed and its filters are registered globally (filter.lfs.clean/smudge/process, required=true), BUT there is NO .gitattributes file anywhere in the repo ? so LFS tracks nothing and is effectively NOT in use. LFS is AVAILABLE if needed but the repo has chosen the 'keep big data out of git' path (output/ gitignored + private OneDrive sync) instead of LFS.",
      "(4) NO EXISTING DATA FIXTURE PRECEDENT: `git ls-files` finds ZERO committed .csv/.jsonl files and no tests/ data/fixture/golden directories. Tests (e.g. test_fusion_golden_telemetry.py) build manifests + synthetic data inline in code with monkeypatch/tmp_path. The ONLY committed structured-data precedent is eval_baselines/*.json ? and those are tiny AGGREGATE metrics (mean_f1, std, per_video score dict), NOT raw per-frame or per-window derived streams.",
      "(5) PROPRIETARY-DATA GUARDRAIL IS EXPLICIT: docs/GOLDEN_DATA_SHARING.md (status 2026-06-15, #175) says the GPU-extracted golden artifacts (trajectory CSVs, *_golden_features.json, manifests) sync via a PRIVATE OneDrive folder at %USERPROFILE%\\OneDrive\\rally-annotator\\golden-data\\ ? OUTSIDE the repo ? precisely so large/proprietary data is 'never committed to git.' Guardrail: 'features derive from the Proprietary corpus ? keep the OneDrive folder private (no public share links)' and 'do not symlink the shared folder into a tracked path.'",
      "(6) SCHEMA/VERSION CONVENTIONS TO MIRROR: DB uses a PRAGMA user_version migration ladder in backend/infrastructure/database.py (currently at v5; ordered `if version < N:` blocks, additive ALTERs that preserve prior rows, tests assert the expected version so drift fails CI). Detector data is DETECTOR-AGNOSTIC: common fields are columns, detector-specific metrics go in a `stats` JSON blob keyed by a `detector` string ? a new detector reuses the table. eval_baselines JSON carries provenance fields (name, description, `computed` repro command, metric values). Golden features JSON has a stable shape: {name, fps, frame_width, candidates:[{start,end,shuttle{5 feats},person{5 feats},label}], gts:[[a,b]]}. Any fixture should likewise carry a version/schema tag + provenance and follow the column-vs-stats-JSON split.",
      "(7) VIDEO PROVENANCE IS CLEAN-BY-DESIGN: docs/GOLDEN_SET_VENDORING.md ? 'Golden sets are built only from footage we license or own ? starting with the founder's personal collection, plus ShuttleSet, plus purpose-recorded clips. No end-user uploads ever flow to a vendor.' So the source is owned/self-recorded + ShuttleSet (broadcast contrast only), NOT third-party broadcast scraping.",
      "(8) MINORS / PRIVACY GUARDRAIL: CLAUDE.md + COMMERCIALIZATION.md C14 + OWNED_MODEL_IMPLEMENTATION_PLAN mandate 'exclude minors' and 'per-user training data needs a separate explicit opt-in.' FIELD_VISIT_PLAYBOOK: 'sessions with minors are not recorded for our library ? adult sessions only.' KHELSUTRA_PRODUCT_OVERVIEW: 'Footage involving minors is never used to improve the tool and is never published.' Any committed fixture must be guaranteed minor-free at source.",
      "(9) LICENSING GUARDRAIL (for completeness): CLAUDE.md ? paid LLMs (Gemini/OpenAI/Anthropic) are inference/serving ONLY, NEVER a training data source; all code+data+weights must be MIT/Apache-2.0/BSD. Derived numeric streams from owned video do not implicate paid-LLM-output rules (they come from MIT-licensed WASB/TrackNet detectors, not from Gemini)."
    ],
    "dataStructures": [
      {
        "name": "Trajectory CSV (per-video shuttle stream)",
        "where": "Produced to gitignored output/ (predict writes <stem>_predict.csv); parsed by backend/pipeline/detectors/tracknet_runner.py::parse_trajectory_csv",
        "shapeOrFormat": "CSV header `Frame,Visibility,X,Y` ? exactly one row per video frame (int frame index, 0/1 visibility, pixel X, pixel Y). Pure numeric, no imagery."

```
      },
      {
        "name": "*_golden_features.json (per-video derived feature record)",
        "where": "backend/eval/fusion_golden.py::extract_video return value; written one JSON
per video to out_dir (output/ or the private OneDrive features/ folder)",
        "shapeOrFormat": "{name, fps, frame_width, candidates:[{start, end, shuttle:{~5
fps/resolution-invariant feats}, person:{~5 person-cue feats}, label:int}],
gts:[[start,end],...]}. The replayable CPU re-scoring substrate; small (a few KB?tens of KB)."
      },
      {
        "name": "eval_baselines/*.json (the ONLY committed structured-data precedent)",
        "where": "eval_baselines/heuristic_lovo_n6.json, gemini_wrapper_2026-06-10.json,
experiment_leaderboard.json ? TRACKED in git",
        "shapeOrFormat": "Tiny aggregate-metric JSON: {name, description, computed:<repro
command>, mean_f1, std, per_video:{video_name: f1, ...}}. Carries provenance; NO per-frame/per-
window data."
      },
      {
        "name": "candidate_segments table (detector-agnostic DB schema)",
        "where": "backend/infrastructure/database.py, migration v1 (PRAGMA user_version=1)",
        "shapeOrFormat": "Columns: id, video_id, detector(str), chunk_index, start_time,
end_time, duration, confidence, stats(JSON for detector-specific metrics),
detection_params(JSON), confirmed_segment_id, human_label, notes, user_id(v4, nullable). Common
fields=columns, detector-specific=stats JSON ? the reuse pattern a fixture should mirror."
      },
      {
        "name": "PRAGMA user_version migration ladder",
        "where": "backend/infrastructure/database.py::_init_db",
        "shapeOrFormat": "Ordered `if version < N:` blocks; currently at v5 (v1
candidate_segments, v2 jobs, v3 self-scheduling, v4 user_id columns, v5 user_models). Additive
ALTERs preserve rows; tests assert expected version (drift fails CI)."
      }
    ],
    "videoDependency": "High but CLEAN. The derived streams (shuttle X/Y trajectories,
candidate-window features, person/audio cues) are extracted FROM the golden videos, so fixtures
encode information about proprietary footage. However: (a) the videos are owned/licensed by the
founder (self-recorded + ShuttleSet), never third-party broadcast scraping and never end-user
uploads (GOLDEN_SET_VENDORING.md); (b) the fixtures are abstract NUMERIC streams (frame index,
pixel coords, scalar features) ? NOT frames/imagery, so they carry no biometric/identifiable
content; (c) the detectors that produce them (WASB/TrackNet) are MIT-licensed and the streams
contain no paid-LLM output, so the licensing guardrail is not implicated. The binding constraint
is the repo's OWN convention (GOLDEN_DATA_SHARING.md) that these proprietary-derived artifacts
live in private OneDrive and are NOT committed ? committing them is a product/privacy reversal
that is owner-gated.",
    "builtVsPlanned": "BUILT: the gitignore rules; eval_baselines/ as the tracked-baseline
pattern; the PRAGMA user_version ladder (v5) with detector-agnostic stats-JSON tables;
backend/eval/fusion_golden.py producing the *_golden_features.json streams; the trajectory CSV
parse path; the test suite's synthetic-data-in-code pattern (no committed data fixtures).
CONVENTION/PLAN (not code-enforced): GOLDEN_DATA_SHARING.md private-OneDrive sync (#175,
status='convention + plan 2026-06-15' ? the optional scripts/sync_golden.py and RALLY_GOLDEN_DIR
env var are NOT required/built yet); the hard train/eval-leakage guardrails
(permissions/approval at ingest+export) are explicitly 'TODO ? design with the user' in
GOLDEN_SET_PHASE1_VIDEOS §5 / BACKLOG; the consent ledger (contribution_consent, minors-
exclusion at the writer) is PERSONALIZATION_PLAN v6 'pending.' git-lfs is available but unused
(no .gitattributes).",
    "constraints": [
      "Committed fixtures = tests/test_*.py (pure/offline/mocked, 70% cov floor) for logic, and
eval_baselines/*.json for tracked metric baselines. Bulk derived data does NOT go in git ?
output/ is gitignored and the golden artifacts are designed to live in private OneDrive, NOT the
repo.",
      "Do NOT commit raw frames/video, *.db, *.pt weights, or large per-frame streams ? all
gitignored by convention. Model artifacts: commit the manifest in artifacts/<name>/<ver>/,
gitignore the weights (ADR-008).",
      "Mirror the detector-agnostic schema: common fields as columns/keys + a `stats`/params
JSON blob keyed by a `detector` string; carry a version/schema tag + provenance (like
```

```
eval_baselines' `computed` repro command and like PRAGMA user_version).",
      "Licensing: every dependency (code+data+weights) MIT/Apache-2.0/BSD only; paid LLMs are
serving-only and NEVER a training/data source ? derived streams must come from the MIT
WASB/TrackNet detectors, not Gemini output.",
      "Privacy: exclude minors at the source; per-user/contributed data needs explicit separate
opt-in + consent ledger; footage involving minors is never used to improve the tool. Committing
derived data from such footage would violate this even though it's 'just numbers.'",
      "Branching safety (CLAUDE.md): branch from origin/master explicitly, re-verify HEAD before
commit, stage explicit paths (never git add -A) ? relevant when actually landing any fixture PR.
NOTE: `git pull --ff-only` failed at session start ('Cannot fast-forward to multiple branches')
? the local checkout may lag origin; treat as possibly stale."
    ],
    "relevantPaths": [
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\CODE_MAP.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_DATA_SHARING.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_VENDORING.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\GOLDEN_SET_PHASE1_VIDEOS.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.gitignore",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\infrastructure\\database.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\fusion_golden.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\tracknet_runner.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\eval_baselines\\heuristic_lovo_n6.json",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\COMMERCIALIZATION.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\OWNED_MODEL_IMPLEMENTATION_PLAN.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\CLAUDE.md",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\tests\\test_fusion_golden_telemetry.py"
    ],
    "risksGaps": [
      "RECOMMENDED DESIGN TO STAY CLEAN: commit only (a) tiny aggregate/metric fixtures and
small DERIVED NUMERIC streams (trajectory CSV rows, feature JSON) ? never frames/imagery; (b)
from videos guaranteed owned/licensed AND minor-free at source; (c) with an explicit
provenance/license/consent tag + a version/schema tag in each fixture. This keeps licensing
clean (MIT detectors, no paid-LLM output) and privacy clean (no biometrics, no minors).",
      "POLICY CONFLICT TO RESOLVE WITH OWNER: GOLDEN_DATA_SHARING.md currently says the
proprietary-derived golden features stay in PRIVATE OneDrive and are NOT committed. A 'commit a
small regression fixture to git' design REVERSES that convention. Because the corpus is labeled
Proprietary, this is a product/privacy decision (owner-gated), not a unilateral engineering
choice ? get explicit sign-off and ideally commit a redacted/synthetic or a tiny owner-cleared
subset rather than the full corpus.",
      "SIZE BUDGET: corpus is n=6 owned golden videos today
(eval_baselines/heuristic_lovo_n6.json + LOVO docs). Per-video trajectory CSV = one row per
frame of `Frame,Visibility,X,Y` (~20-25 bytes/row); a ~2-4 min clip at 30-60fps ? 3.6k-14.4k
rows ? ~80KB-350KB per video ? full n=6 trajectory set ? <2MB. The *_golden_features.json (one
record per candidate window, ~10 feats) is far smaller ? a few KB to tens of KB per video. So a
numeric-only regression fixture for all 6 videos is comfortably git-sized (low single-digit MB);
git-lfs/.gitattributes is NOT needed at this scale.",
      "GAP: no code-enforced guardrail yet prevents derived data from a minors/unconsented or
unlicensed video being committed ? the leakage/consent guards are TODO (PHASE1_VIDEOS §5,
PERSONALIZATION_PLAN v6 pending). Until those exist, the minor-free/owned guarantee rests on
manual discipline; a committed fixture should carry an explicit per-video provenance+consent
assertion.",
      "GAP: artifacts/ is untracked and NOT listed in .gitignore ? if a fixture-generating PR
also touches artifacts/, be careful not to accidentally stage weights; stage explicit paths
only.",
      "STALENESS FLAG: `git pull --ff-only` failed ('Cannot fast-forward to multiple branches')
? local master may lag origin/master; re-fetch and branch from origin/master before landing any
fixture PR (per CLAUDE.md shared-checkout safety)."
    ]
```

```
  }
]
```

2. assistant (2026-06-16T02:28:05.104Z)

I'll analyze the codebase to ground my design proposal. Let me start by pulling the latest and examining the key seams the grounding identified.

3. user (2026-06-16T02:28:08.484Z)

```
master
b6a39c4 feat(runner): fully-local no-AI segmenter mode (+ WASB stream flag, .env load) (#183)
```

4. user (2026-06-16T02:28:08.489Z)

```
1       """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4       inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6           1. Translate the Windows video path to its WSL-visible form (and optionally
7              stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8           2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9           3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10          4. Convert that trajectory into high-motion "action windows" ? the same
11             candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12             pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
31      class TrackNetConfig:
32          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "TrackNetV4"
37          weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
38          queue_length: int = 5
39          stage_in_wsl: bool = True                       # copy video into WSL fs first
(perf)
40          wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
41          timeout_sec: int = 1800                         # 30 min; kills a hung call (0 = no
timeout)
42          keep_frames: bool = False                       # retain staged video after success
(debug only)
```

```
        min_free_gb: float = 0.0                        # warn if WSL free space below this
(0 = off)

        @classmethod
        def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
            tn = (idx_cfg or {}).get("tracknet", {}) or {}
            return cls(
                distro=tn.get("wsl_distro", "Ubuntu"),
                repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
                conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
                conda_env=tn.get("conda_env", "TrackNetV4"),
                weights_path=tn.get("weights_path", ""),
                queue_length=int(tn.get("queue_length", 5)),
                stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
                wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
                timeout_sec=int(tn.get("timeout_sec", 1800)),
                keep_frames=bool(tn.get("keep_frames", False)),
                min_free_gb=float(tn.get("min_free_gb", 0.0)),
            )


    def to_wsl_mnt_path(win_path: str) -> str:
        """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).

        Already-POSIX paths (starting with / or ~) are returned unchanged so the
        same helper is safe to call on either kind of path.
        """
        p = str(win_path)
        if p.startswith("/") or p.startswith("~"):
            return p
        p = p.replace("\\", "/")
        if len(p) >= 2 and p[1] == ":":
            drive = p[0].lower()
            return f"/mnt/{drive}{p[2:]}"
        return p


    @dataclass
    class TrajectoryPoint:
        frame: int
        visible: bool
        x: float
        y: float


    class TrackNetRunner(WslCommandMixin, DetectorRunner):
        """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.

        Implements the common DetectorRunner contract so a future Linux-native or
        alternative trajectory runner can be substituted without touching the hybrids.
        """

        def __init__(self, cfg: TrackNetConfig):
            self.cfg = cfg

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.

            Returns (ok, message). Cheap to call before a long run.
            """
            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"python -c \"import tensorflow as tf; "
                f"print('TF', tf.__version__, 'GPUs',
len(tf.config.list_physical_devices('GPU')))\""
```

```
            )
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? is this running on Windows with WSL2
installed?"
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL env check failed: {(res.stderr or out).strip()}"
            return True, out

        # ------------------------------------------------------------------ inference

        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.

            ``output_win_dir`` is a Windows directory (under the repo's output/) that
            WSL writes into via its /mnt view, so the Windows process can read the CSV
            back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
            therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
            a filename cannot collide on the output CSV.
            """
            if not self.cfg.weights_path:
                logger.error(
                    "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
                    "in config.json to the WSL path of your trained TrackNetV4 weights."
                )
                return None

            # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
            # relative paths through unchanged, so WSL would resolve them against the
            # predict.py cwd instead of the repo (same bug class as wasb_runner).
            output_win_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize separators so basename/splitext work when tests (or collector)
            # pass Windows-style paths on a Linux host.
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)

            # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
            if ext.lower() not in (".mp4", ".avi"):
                logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
                return None

            wsl_out = to_wsl_mnt_path(output_win_dir)
            # Namespace by video_id so two same-filename videos don't collide on the CSV.
            # predict.py derives its output name from the (staged) video stem, so staging
            # under the namespaced name propagates into "<key>_predict.csv".
            key = f"{stem}__{video_id[:12]}" if video_id else stem
            expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")

            # Resolve the video path WSL will read.
            if self.cfg.stage_in_wsl:
                staged = self._stage_video(video_win_path, f"{key}{ext}")
                if staged is None:
                    return None
                wsl_video = staged
            else:
                # No staging: predict.py reads /mnt directly and writes
"<stem>_predict.csv".
```

```
                # We cannot rename its output, so fall back to the un-namespaced name.
                wsl_video = to_wsl_mnt_path(video_win_path)
                expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")

            if self.cfg.min_free_gb > 0:
                free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
                if free is not None and free < self.cfg.min_free_gb:
                    logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
                                   "consider running tools/cleanup_caches.py.",
                                   free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)

            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"cd {self.cfg.repo_dir}/src && "
                f"python predict.py "
                f"--video_path {wsl_tilde_quote(wsl_video)} "
                f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
                f"--output_dir {wsl_tilde_quote(wsl_out)} "
                f"--queue_length {self.cfg.queue_length}"
            )
            logger.info("Running TrackNet inference on %s ...", base)
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
                return None

            if res.returncode != 0:
                logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
                return None

            if not os.path.exists(expected_csv_win):
                logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
                return None
            logger.info("TrackNet trajectory CSV: %s", expected_csv_win)

            # Success ? the CSV is the durable output; drop the staged video copy (a pure,
            # regenerable intermediate). On earlier failure we return above without
cleaning.
            if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
                logger.info("Cache hygiene: removing staged video for %s "
                            "(set indexing.tracknet.keep_frames=true to retain).", key)
                self._wsl_rm_rf([wsl_video])
            return expected_csv_win

        def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
            """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).

            Returns the WSL path to the staged copy, or None on failure.
            """
            src_mnt = to_wsl_mnt_path(video_win_path)
            reason = self.wsl_source_unreadable_reason(src_mnt)
            if reason:
                logger.error("Cannot stage video into WSL: %s", reason)
                return None
            dest = f"{self.cfg.wsl_stage_dir}/{base}"
            cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("Staging video into WSL timed out.")
```

```
                return None
            if res.returncode != 0 or "OK" not in (res.stdout or ""):
                logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
                return None
            return dest

        # ------------------------------------------------------------ CSV -> windows

        @staticmethod
        def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
            """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
            points: List[TrajectoryPoint] = []
            with open(csv_win_path, newline="") as f:
                reader = csv.DictReader(f)
                for row in reader:
                    try:
                        points.append(TrajectoryPoint(
                            frame=int(row["Frame"]),
                            visible=int(row["Visibility"]) == 1,
                            x=float(row["X"]),
                            y=float(row["Y"]),
                        ))
                    except (KeyError, ValueError):
                        continue
            points.sort(key=lambda p: p.frame)
            return points

        @staticmethod
        def trajectory_to_action_windows(
            points: List[TrajectoryPoint],
            fps: float,
            frame_width: float,
            chunk_start: float = 0.0,
            velocity_thresh: float = 0.02,
            merge_gap: float = 2.0,
            min_window_duration: float = 2.0,
            chunk_index: Optional[int] = None,
        ) -> List[Dict[str, Any]]:
            """Convert a shuttle trajectory into candidate rally windows.

            A frame is "active" when the shuttle is visible AND its inter-frame
            displacement (normalised by frame width, per second) exceeds
            ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
            ``merge_gap`` seconds of each other are grouped into one window; short
            windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
            shape feeding the AI-handoff phase is identical.
            """
            if fps <= 0:
                fps = 30.0
            if frame_width <= 0:
                frame_width = 1280.0

            # Compute per-frame normalised velocity from the last *visible* point.
            active = []  # (frame_idx, velocity)
            prev: Optional[TrajectoryPoint] = None
            for p in points:
                if not p.visible:
                    prev = None  # break the trajectory; absent shuttle = not in flight
                    continue
                if prev is not None:
                    dframes = p.frame - prev.frame
                    if dframes > 0:
                        dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
                        velocity = (dist / frame_width) * (fps / dframes)
```

```
                        if velocity > velocity_thresh:
                            active.append((p.frame, velocity))
                prev = p

            if not active:
                return []

            max_gap_frames = int(merge_gap * fps)

            def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
                vels = [v for _, v in group]
                return {
                    "start": group[0][0] / fps + chunk_start,
                    "end": group[-1][0] / fps + chunk_start,
                    "active_frame_count": len(group),
                    "peak_velocity": max(vels),
                    "mean_velocity": sum(vels) / len(vels),
                    "track_id_count": 1,  # single shuttle; kept for window-shape parity
                    "chunk_index": chunk_index,
                }

            windows = []
            group = [active[0]]
            for af in active[1:]:
                if af[0] - group[-1][0] <= max_gap_frames:
                    group.append(af)
                else:
                    windows.append(_finalize(group))
                    group = [af]
            windows.append(_finalize(group))

            return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

```

5. assistant (2026-06-16T02:28:11.377Z)

Let me examine the core scoring/regression seams and the existing fixture-loading patterns.

6. user (2026-06-16T02:28:12.402Z)

```
"""Regression runner ? verify the production tool against golden-set ground truth.

Implements the explicit ask: *"given a video, get a golden set created and verify
its precision and recall against the production tool."* This is deliberately a thin
wrapper over machinery that already exists:

    ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
    predictions   ? the production pipeline's stored DB rows     [backend.eval.harness]
    comparison    ? interval matching + P/R/F1                   [backend.eval.metrics]
    baseline      ? a per-video snapshot of "known good" numbers [this module]

A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
"""
import os
import json
import hashlib
import logging
from dataclasses import dataclass, asdict
from typing import Dict, List, Optional

from backend.infrastructure.database import Database
from backend.eval import metrics
```

```
    from backend.eval.harness import get_predictions
    from backend.annotations import annotation_registry

    logger = logging.getLogger(__name__)

    # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
    # actually load a committed baseline to compare against.
    DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
    DEFAULT_TOLERANCE = 0.05


    def gt_hash(ground_truth: List[metrics.Interval]) -> str:
        """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
        norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
        return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]


    @dataclass
    class RegressionResult:
        video_id: str
        stage: str
        iou_threshold: float
        precision: float
        recall: float
        f1: float
        # Baseline values compared against (None when no baseline existed yet).
        baseline_precision: Optional[float]
        baseline_recall: Optional[float]
        tolerance: float
        regressed: bool
        reasons: List[str]
        # True only when a baseline existed to compare against. Distinguishes a genuine PASS
        # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
        has_baseline: bool = False
        gt_hash: Optional[str] = None

        def as_dict(self) -> dict:
            return asdict(self)


    # ---------------------------------------------------------------- baseline store

    def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
        """Load the per-video baseline snapshot file (returns {} if absent)."""
        if not os.path.isfile(path):
            return {}
        with open(path) as f:
            return json.load(f)


    def _baseline_key(video_id: str, stage: str) -> str:
        return f"{video_id}::{stage}"


    def save_baseline(video_id: str, stage: str, precision: float, recall: float,
                      f1: float, path: str = DEFAULT_BASELINE_PATH,
                      iou_threshold: Optional[float] = None, detector: Optional[str] = None,
                      gt_fingerprint: Optional[str] = None) -> None:
        """Snapshot a video/stage's current numbers as the new 'known good' baseline.

        Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
        later run computed under different settings (or against different labels) is
detected
```

```
88          as incommensurable rather than silently compared.
89          """
90          baselines = load_baselines(path)
91          baselines[_baseline_key(video_id, stage)] = {
92              "video_id": video_id, "stage": stage,
93              "precision": precision, "recall": recall, "f1": f1,
94              "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95          }
96          os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97          with open(path, "w") as f:
98              json.dump(baselines, f, indent=2, sort_keys=True)
99          logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
_baseline_key(video_id, stage), precision, recall)
100
101
102     # ---------------------------------------------------------------- the runner
103
104     def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105                 stage: str = "final", iou_threshold: float = 0.5,
106                 detector: Optional[str] = None,
107                 tolerance: float = DEFAULT_TOLERANCE,
108                 baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109         """Score stored predictions for one video against ground truth and compare to
baseline.
110
111         `ground_truth` is a list of (start, end) intervals ? typically obtained from an
112         AnnotationProvider (see `regress_with_provider`). A regression is flagged when
113         precision OR recall falls more than `tolerance` below the snapshotted baseline.
114         If no baseline exists yet, `regressed` is False (nothing to compare to) and the
115         caller can choose to snapshot the current numbers via `save_baseline`.
116         """
117         preds = get_predictions(db, video_id, stage, detector=detector)
118         s = metrics.score(preds, ground_truth, iou_threshold)
119         fp = gt_hash(ground_truth)
120
121         baselines = load_baselines(baseline_path)
122         b = baselines.get(_baseline_key(video_id, stage))
123
124         reasons: List[str] = []
125         regressed = False
126         has_baseline = b is not None
127         base_p = base_r = None
128         if b is not None:
129             base_p, base_r = b["precision"], b["recall"]
130             # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
131             # against different labels is not a valid reference ? flag it instead of
trusting it.
132             b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
133             if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
134                 regressed = True
135                 reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
136             if b_det is not None and b_det != detector:
137                 regressed = True
138                 reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
139             if b_hash is not None and b_hash != fp:
140                 regressed = True
141                 reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142             if s.precision < base_p - tolerance:
143                 regressed = True
144                 reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
```

```
{tolerance}")
            if s.recall < base_r - tolerance:
                regressed = True
                reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
        else:
            reasons.append("no baseline recorded for this video/stage ? nothing to compare")

        return RegressionResult(
            video_id=video_id, stage=stage, iou_threshold=iou_threshold,
            precision=s.precision, recall=s.recall, f1=s.f1,
            baseline_precision=base_p, baseline_recall=base_r,
            tolerance=tolerance, regressed=regressed, reasons=reasons,
            has_baseline=has_baseline, gt_hash=fp,
        )


    def regress_with_provider(db: Database, video_id: str, video_ref: str,
                              tier: str = "file", stage: str = "final",
                              iou_threshold: float = 0.5,
                              detector: Optional[str] = None,
                              tolerance: float = DEFAULT_TOLERANCE,
                              baseline_path: str = DEFAULT_BASELINE_PATH,
                              guideline: Optional[str] = None,
                              sport: str = "badminton",
                              provider_kwargs: Optional[dict] = None) -> RegressionResult:
        """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.

        `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
        once those providers land). `video_ref` is whatever that provider expects (a CSV
        path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
        predictions in the DB (the file MD5).
        """
        provider = annotation_registry.get(tier, **(provider_kwargs or {}))
        gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
        return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
                       detector=detector, tolerance=tolerance, baseline_path=baseline_path)


    def format_result(r: RegressionResult) -> str:
        verdict = "REGRESSED" if r.regressed else "ok"
        head = (f"[{verdict}] {r.video_id} ({r.stage}) "
                f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
        if r.baseline_precision is not None:
            head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
        return head + "".join(f"\n   - {why}" for why in r.reasons)

```

7. user (2026-06-16T02:28:12.881Z)

```
"""Pure temporal interval-matching metrics for rally evaluation.

No I/O, no DB, no global state ? every function here is deterministic and unit
tested. An "interval" is a `(start, end)` tuple of floats in seconds with
start < end. We score a list of *predicted* rally intervals against a list of
*ground-truth* intervals via temporal IoU and one-to-one greedy matching.
"""

from dataclasses import dataclass, field
from typing import List, Tuple, Optional

Interval = Tuple[float, float]

```

```

    def interval_iou(a: Interval, b: Interval) -> float:
        """Temporal intersection-over-union of two intervals.

        Returns 0.0 when they don't overlap (or either is degenerate). IoU is
        overlap / union, both measured as durations on the timeline.
        """
        a_start, a_end = a
        b_start, b_end = b
        if a_end <= a_start or b_end <= b_start:
            return 0.0
        inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
        if inter <= 0.0:
            return 0.0
        union = (a_end - a_start) + (b_end - b_start) - inter
        return inter / union if union > 0 else 0.0


    @dataclass
    class Match:
        """One matched predicted?ground-truth pair."""
        pred_index: int
        gt_index: int
        iou: float


    @dataclass
    class MatchResult:
        """Outcome of matching predictions to ground truth at a given IoU threshold."""
        matches: List[Match] = field(default_factory=list)
        unmatched_pred_indices: List[int] = field(default_factory=list)  # false positives
        unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives
(misses)


    def match_intervals(preds: List[Interval], gts: List[Interval],
                        iou_threshold: float = 0.5) -> MatchResult:
        """Greedy one-to-one matching of predictions to ground truth by descending IoU.

        Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
        are claimed highest-IoU first, and each prediction/GT can be used once. This
        keeps a single prediction from being credited for two ground-truth rallies
        (and vice versa).
        """
        candidates = []
        for pi, p in enumerate(preds):
            for gi, g in enumerate(gts):
                iou = interval_iou(p, g)
                # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
would
                # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
                if iou > 0.0 and iou >= iou_threshold:
                    candidates.append((iou, pi, gi))
        # Highest IoU first; ties broken by indices for determinism.
        candidates.sort(key=lambda c: (-c[0], c[1], c[2]))

        used_pred, used_gt = set(), set()
        matches: List[Match] = []
        for iou, pi, gi in candidates:
            if pi in used_pred or gi in used_gt:
                continue
            used_pred.add(pi)
            used_gt.add(gi)
            matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))

```

```
        unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
        unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
        return MatchResult(matches=matches,
                           unmatched_pred_indices=unmatched_pred,
                           unmatched_gt_indices=unmatched_gt)


    @dataclass
    class Score:
        """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
        iou_threshold: float
        num_pred: int
        num_gt: int
        true_positives: int
        false_positives: int
        false_negatives: int
        precision: float
        recall: float
        f1: float
        mean_iou: float            # over matched pairs (0.0 if none)
        mean_start_error: float    # mean |pred_start - gt_start| over matches, seconds
        mean_end_error: float      # mean |pred_end - gt_end| over matches, seconds

        def as_dict(self) -> dict:
            return self.__dict__.copy()


    def score(preds: List[Interval], gts: List[Interval],
              iou_threshold: float = 0.5,
              match_result: Optional[MatchResult] = None) -> Score:
        """Compute precision/recall/F1, mean matched IoU, and boundary errors.

        Pass a precomputed `match_result` to avoid re-matching (e.g. when also
        extracting the failure lists); otherwise matching is done here.
        """
        mr = match_result if match_result is not None else match_intervals(preds, gts,
iou_threshold)

        tp = len(mr.matches)
        fp = len(mr.unmatched_pred_indices)
        fn = len(mr.unmatched_gt_indices)

        precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
        f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
else 0.0

        if mr.matches:
            mean_iou = sum(m.iou for m in mr.matches) / tp
            mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
mr.matches) / tp
            mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
mr.matches) / tp
        else:
            mean_iou = mean_start_err = mean_end_err = 0.0

        return Score(
            iou_threshold=iou_threshold,
            num_pred=len(preds), num_gt=len(gts),
            true_positives=tp, false_positives=fp, false_negatives=fn,
            precision=precision, recall=recall, f1=f1,
            mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
        )


```

```
138     def pr_curve(preds: List[Interval], gts: List[Interval],
139                  thresholds: Optional[List[float]] = None) -> List[Score]:
140         """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
141
142         Lets you see strict-vs-lenient behaviour rather than a single operating point.
143         """
144         if thresholds is None:
145             thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
146         return [score(preds, gts, iou_threshold=t) for t in thresholds]
147
```

8. assistant (2026-06-16T02:28:16.498Z)

Now let me look at the existing per-video regression test (the template to mirror), the calibrate_wasb seam that already replays trajectory CSVs, and the fusion_compare loader.

9. user (2026-06-16T02:28:17.471Z)

```
1       """SERVED Gate-A regression litmus ? the *production* FusionSegmenter path on real
footage.
2
3       The flagged gap (#146): the served default flip (``default_segmenter=fusion_hybrid`` +
4       ``min_crossings=2``) is **mock-tested only** on real video (``test_fusion_hybrid.py``
drives a
5       MagicMock runner). The offline ablation litmus
(``test_ablation.test_litmus_reproduces_gate_a``)
6       validates the Gate-A *signal* ? but on the **permissive proposal windowing**
7       (``distill_local.PROPOSAL``), NOT the production operating point. This module closes the
gap: it
8       runs the real ``FusionSegmenter`` decision seams (``_enrich_candidate_stats`` net-
crossings +
9       ``_select_for_handoff`` Gate A) over windows from the **production**
10      ``trajectory_to_action_windows`` config, on the 6 cached golden WASB trajectories, and
asserts
11      the served path's measured behaviour.
12
13      GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips
cleanly in
14      CI where the golden trajectories are absent; runs on the owner box where they are
present.
15
16      HONEST FINDING baked into the assertions (2026-06-14): on the **production windowing**
Gate A is
17      NOT the +0.074 win the offline harness reports. Production windowing is already
conservative
18      (velocity_threshold 0.02, merge_gap 2.0, min_window 2.0) ? few long windows, over-seg
ratio
19      already < 1 ? there is little held-shuttle junk left for Gate A to cut. So the served
?F1 is
20      ~neutral (slightly negative), NOT +0.074. The +0.074 reproduces ONLY under the
permissive
21      proposal windowing (asserted in ``test_proposal_windowing_reproduces_offline_lift``
below ? that
22      is what the offline litmus measures). These tests pin BOTH facts so a future windowing
change
23      that silently revives ? or breaks ? either one trips the litmus.
24      """
25      from __future__ import annotations
26
27      import json
28      import os
29
30      import pytest
31
32      # cv2 is imported transitively by trajectory_hybrid (the served engine). On a box
```

```
without it
33      # (CI), skip the whole module rather than erroring at import.
34      pytest.importorskip("cv2")
35
36      from backend.eval import served_gate_a as sga  # noqa: E402
37      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner  # noqa: E402
38      from backend.pipeline.detectors import rally_gate  # noqa: E402
39      from backend.eval.calibrate_wasb import load_golden_gts  # noqa: E402
40      from backend.eval.metrics import score  # noqa: E402
41      from backend.eval import segmentation_metrics as _segm  # noqa: E402
42
43      # The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a
fresh
44      # worktree / CI). Resolve it relative to this repo root first, with an env override for
an
45      # out-of-tree checkout.
46      _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
47      MANIFEST = os.environ.get(
48          "SERVED_GATE_A_MANIFEST",
49          os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
50      )
51
52
53      def _golden_specs():
54          """Return the manifest specs whose trajectory + label files are all present, else
[]."""
55          if not os.path.isfile(MANIFEST):
56              return []
57          with open(MANIFEST, encoding="utf-8") as f:
58              specs = json.load(f)
59          ok = [s for s in specs
60                if os.path.isfile(str(s.get("trajectory", "")))
61                and os.path.isfile(str(s.get("labels", "")))]
62          return ok if len(ok) >= 6 else []
63
64
65      _SPECS = _golden_specs()
66      _skip = pytest.mark.skipif(
67          not _SPECS,
68          reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
69      )
70
71
72      # ---------------------------------------------------------------------- served-path
litmus
73
74      @_skip
75      def test_served_path_runs_on_all_six_golden_videos():
76          """The production FusionSegmenter decision path runs end-to-end (no GPU/Gemini) on
all 6
77          golden trajectories and yields a handoff window list for each, for both gate
settings."""
78          results = sga.evaluate_manifest(_SPECS)
79          assert len(results) == 6
80          for r in results:
81              assert r.num_gt > 0
82              assert r.n_handoff_off >= r.n_handoff_on   # Gate A only ever drops windows,
never adds
83
84
85      @_skip
86      def test_served_gate_a_does_not_regress_f1():
87          """SERVED no-regression guard: on the production windowing Gate A must not
MATERIALLY hurt
88          F1. It is ~neutral here (the held-shuttle junk the offline harness sees is already
```

```
merged
89          away by the conservative production windowing) ? so we assert the mean ?F1 sits in a
tight
90          band around 0, never a real regression. (The +0.074 lift lives under the permissive
proposal
91          windowing ? see the dedicated test below.)"""
92          results = sga.evaluate_manifest(_SPECS)
93          agg = sga.aggregate(results)
94          # Honest measured value (2026-06-14): mean served ?F1 ? -0.008. Pin a tolerance band
so a
95          # real regression (Gate A actively destroying served F1) trips, while the measured
~neutral
96          # result passes.
97          assert -0.03 <= agg["mean_delta_f1"] <= 0.03, agg
98          # And Gate A must never collapse served F1 to ~0 on the production path.
99          assert agg["mean_f1_on"] >= 0.5 * agg["mean_f1_off"] - 1e-9, agg
100
101
102     @_skip
103     def test_served_gate_a_does_not_increase_over_segmentation():
104         """Gate A only ever DROPS windows, so the served over-seg ratio must not rise when
it's on
105         (the direction-of-effect invariant ? even where the absolute lift is ~neutral)."""
106         results = sga.evaluate_manifest(_SPECS)
107         agg = sga.aggregate(results)
108         assert agg["mean_seg_ratio_on"] <= agg["mean_seg_ratio_off"] + 1e-9, agg
109         # Per-video too: the gate is monotone (kept windows ? all windows).
110         for r in results:
111             assert r.seg_ratio_on <= r.seg_ratio_off + 1e-9, r
112
113
114     @_skip
115     def test_served_uses_production_windowing_operating_point():
116         """Pin the served operating point to config.json's indexing defaults ? if someone
retunes
117         production windowing, this litmus's premise (and the divergence it documents) is
revisited
118         deliberately, not silently."""
119         assert sga.PROD_VELOCITY_THRESH == 0.02
120         assert sga.PROD_MERGE_GAP == 2.0
121         assert sga.PROD_MIN_WINDOW == 2.0
122
123
124     # ------------------------------------------------ the +0.074 lives under PROPOSAL
windowing
125
126     # distill_local.PROPOSAL ? the permissive, recall-oriented candidate windowing the
OFFLINE
127     # ablation/fusion_compare harness builds on (low velocity thresh, short merge/min-dur ?
many
128     # small windows). The +0.074 Gate-A lift is a property of THIS windowing, not the served
one.
129     _PROPOSAL = (0.005, 1.0, 0.5)
130
131
132     @_skip
133     def test_proposal_windowing_reproduces_offline_lift():
134         """CONTROL / root-cause pin: under the OFFLINE harness's permissive proposal
windowing, the
135         SAME ``rally_gate`` net-crossing gate the served seam uses reproduces the offline
litmus
136         (?F1 ? +0.074, 6/6 wins, over-seg 1.68?1.01). This proves the served ~neutral result
is a
137         WINDOWING difference, not a broken gate ? and locks the offline number to its true
premise."""
```

```
        d_on, d_off, seg_on, seg_off = [], [], [], []
        for s in _SPECS:
            fps, fw = float(s["fps"]), float(s["frame_width"])
            pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
            wins = TrackNetRunner.trajectory_to_action_windows(
                pts, fps=fps, frame_width=fw, velocity_thresh=_PROPOSAL[0],
                merge_gap=_PROPOSAL[1], min_window_duration=_PROPOSAL[2])
            iv = [(w["start"], w["end"]) for w in wins]
            est = rally_gate.estimate_play_axis(pts)
            gated = rally_gate.gate_windows(pts, iv, fps, min_crossings=0, estimate=est)
            on = [(g.start, g.end) for g in gated if g.crossings >= 2]
            gts = load_golden_gts(s["labels"])
            d_on.append(score(on, gts).f1)
            d_off.append(score(iv, gts).f1)
            seg_on.append(_segm.segment_count_ratio(on, gts))
            seg_off.append(_segm.segment_count_ratio(iv, gts))
        n = len(d_on)
        mean_delta = (sum(d_on) - sum(d_off)) / n
        wins = sum(1 for a, b in zip(d_on, d_off) if a > b + 1e-9)
        assert mean_delta == pytest.approx(0.074, abs=0.01), mean_delta
        assert wins == 6, wins
        assert sum(seg_off) / n == pytest.approx(1.68, abs=0.05)
        assert sum(seg_on) / n == pytest.approx(1.01, abs=0.05)

```

10. user (2026-06-16T02:28:17.960Z)

```
"""Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".

Wires the already-tested pieces together:
  WASB trajectory CSV (Frame,Visibility,X,Y)
    -> TrackNetRunner.trajectory_to_action_windows(cfg)   [predict_fn]
    -> backend.eval.calibration (improvement_report / cross_validate)

Run (after a full-video WASB pass exists):
    python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
        --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920

Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
same GT is an **optimistic** upper bound. The trustworthy number is the
cross-validated held-out **recall** (and, once a 2nd labelled video exists,
`leave_one_video_out` for precision/F1 generalization).
"""
from __future__ import annotations

import argparse
from typing import List, Tuple, Callable

from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
from backend.eval.calibration import (
    select_best, improvement_report, cross_validate, evaluate,
)

Interval = Tuple[float, float]
Config = Tuple[float, float, float]   # (velocity_thresh, merge_gap,
min_window_duration)

BASELINE: Config = (0.02, 2.0, 2.0)   # the WasbConfig / trajectory_to_action_windows
defaults


def load_golden_gts(csv_path: str) -> List[Interval]:
    """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.

```

```
        Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
        start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
        driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
        """
        from backend.eval.gt_loader import load_gt_intervals
        return load_gt_intervals(csv_path, strict=False)


    def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
        """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)

        def predict_fn(cfg: Config) -> List[Interval]:
            vt, mg, mwd = cfg
            wins = TrackNetRunner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width,
                velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
            )
            return [(w["start"], w["end"]) for w in wins]

        return predict_fn


    def default_grid() -> List[Config]:
        return [(vt, mg, mwd)
                for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
                for mg in (1.0, 2.0, 3.0)
                for mwd in (1.0, 2.0, 3.0)]


    def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
            iou: float = 0.5) -> dict:
        gts = load_golden_gts(labels_csv)
        if not gts:
            raise SystemExit(f"no usable golden rallies in {labels_csv}")
        predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
        grid = default_grid()

        base = evaluate(predict_fn(BASELINE), gts, iou)
        best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
iou_threshold=iou)
        fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou)   # OPTIMISTIC
(fit on full GT)
        cv = cross_validate(predict_fn, grid, gts, k=5)                      # HONEST held-
out recall

        print("=== WASB rally-segmentation calibration ===")
        print(f"golden rallies: {len(gts)}  | IoU thresh: {iou}")
        print(f"baseline cfg {BASELINE}: precision={base.precision:.3f}
recall={base.recall:.3f} "
              f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
        print(f"tuned cfg    {best_cfg}: f1={best_f1:.3f}")
        print(f"  precision {fit['baseline']['precision']:.3f} ->
{fit['tuned']['precision']:.3f} "
              f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)")
        print(f"  f1        {fit['baseline']['f1']:.3f} -> {fit['tuned']['f1']:.3f} "
              f"({fit['f1_delta_pct']:+.1f}%)")
        print(f"HONEST (cross-val, {cv.get('folds','?')} folds): held-out recall "
              f"{cv.get('mean_test',float('nan')):.3f} ± {cv.get('std_test',0):.3f}, "
              f"generalization_gap {cv.get('generalization_gap',float('nan')):+.3f}")
        print("(For precision/F1 generalization, add a 2nd labelled video ->
leave_one_video_out.)")
        return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}

```

```
94
95      def main():
96          ap = argparse.ArgumentParser()
97          ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
98          ap.add_argument("--labels", required=True, help="golden labels CSV
(start_time/end_time cols)")
99          ap.add_argument("--fps", type=float, default=59.94)
100         ap.add_argument("--frame-width", type=float, default=1920.0)
101         ap.add_argument("--iou", type=float, default=0.5)
102         args = ap.parse_args()
103         run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)
104
105
106     if __name__ == "__main__":
107         main()
108
```

11. assistant (2026-06-16T02:28:20.619Z)

Now let me look at the gt_loader (the canonical CSV reader), the .gitignore, and the CI workflow to understand the determinism and CI gate landscape.

12. user (2026-06-16T02:28:22.041Z)

```
1       """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3       The repo historically carried two incompatible loaders:
4       - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5         ``curate export`` format), raises on bad rows;
6       - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7         (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9       A file in one convention fed to the other loader yielded zero/garbage intervals with no
10      error. This module accepts BOTH conventions (header-mapped, else positional), so every
11      quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12      flows through one reader. Both legacy entry points now delegate here.
13
14      Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15      - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16        columns ? rally_number, ending_reason, sport, ? ? are ignored);
17      - no header: columns 0,1 positionally.
18      Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
(``annotations.parse_timestamp``).
19      """
20      import csv
21      import logging
22      from typing import List, Optional, Tuple
23
24      from backend.eval.annotations import parse_timestamp
25
26      logger = logging.getLogger(__name__)
27
28      Interval = Tuple[float, float]
29
30      _START_NAMES = ("start", "start_time")
31      _END_NAMES = ("end", "end_time")
32
33
34      def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35          """If `row` is a header naming start/end columns, return their indices, else
None."""
36          cells = [(c or "").strip().lower() for c in row]
37          start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
```

```
        end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
        if start_idx is not None and end_idx is not None:
            return start_idx, end_idx
        return None


    def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
        """Load GT rally intervals from either golden-CSV convention.

        strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
        (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
        strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
        skipped ? a silently shrinking GT set flatters precision.

        Returns intervals sorted by start; overlapping GT intervals are logged as a warning
        (they break one-to-one matching and usually indicate a labeling error).
        """
        intervals: List[Interval] = []
        skipped = 0
        cols: Optional[Tuple[int, int]] = None   # (start_idx, end_idx); None until detected
        saw_header = False

        with open(csv_path, newline="", encoding="utf-8-sig") as f:
            for lineno, row in enumerate(csv.reader(f), start=1):
                if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
                    continue
                if cols is None and not saw_header:
                    header = _header_columns(row)
                    if header is not None:
                        cols = header
                        saw_header = True
                        continue            # consume the header row
                    cols = (0, 1)           # headerless: positional start,end
                assert cols is not None     # every branch above assigns it
                si, ei = cols
                try:
                    if len(row) <= max(si, ei):
                        raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
                    start = parse_timestamp(row[si])
                    end = parse_timestamp(row[ei])
                    if start < 0:
                        raise ValueError(f"negative start ({start})")
                    if start >= end:
                        raise ValueError(f"start ({start}) >= end ({end})")
                except ValueError as e:
                    if strict:
                        raise ValueError(f"{csv_path}:{lineno}: {e}") from e
                    skipped += 1
                    continue
                intervals.append((start, end))

        if skipped:
            logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)

        intervals.sort(key=lambda iv: iv[0])
        for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
            if s2 < e1:
                logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
                               "labeling error? Overlaps break one-to-one matching.",
                               csv_path, s1, e1, s2, e2)
        return intervals

```

13. user (2026-06-16T02:28:22.515Z)

```
1       name: CI
2
3       on:
4         push:
5           branches: [master]
6         pull_request:
7
8       permissions:
9         contents: read
10
11      concurrency:
12        group: ci-${{ github.ref }}
13        cancel-in-progress: true
14
15      jobs:
16        # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17        # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18        # tracked .py, including scripts/ and training/, which the backend-only
19        # py_compile smoke sweep does not reach.
20        lint:
21          name: Ruff lint
22          runs-on: ubuntu-latest
23          steps:
24            - uses: actions/checkout@v6
25            - uses: actions/setup-python@v6
26              with:
27                python-version: "3.13"
28            - name: Install ruff
29              run: python -m pip install ruff
30            - name: Ruff check
31              run: ruff check . --output-format github
32
33        # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
34        # per-module quarantine list ? ratchet by removing entries as modules clean
35        # up. Typed core deps are installed so mypy sees their real signatures
36        # (without them ignore_missing_imports would Any-stub half the checks).
37        typecheck:
38          name: Mypy (lenient baseline)
39          runs-on: ubuntu-latest
40          steps:
41            - uses: actions/checkout@v6
42            - uses: actions/setup-python@v6
43              with:
44                python-version: "3.13"
45            - name: Install mypy + typed core deps
46              run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
47            - name: Mypy
48              run: mypy backend training
49
50        # Fast guard against the 7fbbeb0 class of breakage: a SyntaxError or
51        # module-level import error shipping while the suite stays green.
52        # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
53        # on machines without the GPU stack (test_import_smoke stubs what's absent).
54        import-smoke:
55          name: Syntax sweep + import smoke (no GPU stack)
56          runs-on: ubuntu-latest
57          steps:
58            - uses: actions/checkout@v6
59            - uses: actions/setup-python@v6
60              with:
61                python-version: "3.13"
62            - name: Install minimal deps
```

```
63              run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
64            - name: Run import smoke tests
65              run: python -m pytest tests/test_import_smoke.py -v
66
67        full-suite:
68          name: Full test suite (offline, mocked)
69          runs-on: ubuntu-latest
70          steps:
71            - uses: actions/checkout@v6
72            - uses: actions/setup-python@v6
73              with:
74                python-version: "3.13"
75                cache: pip
76            - name: System libs for opencv-python
77              run: |
78                sudo apt-get update
79                sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
80            - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
81              # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
82              # need an out-of-band index that PEP 621 can't express. Keep installing
83              # them separately here, before the editable install pulls the rest.
84              run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
85            - name: Install package (editable) + dev tooling
86              # Editable PEP 621 install: runtime deps + the `dev` group
87              # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
88              run: python -m pip install -e .[dev]
89            # obsreport (private, soft dep) is absent here: reporting tests skip via
90            # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
91            # is therefore calibrated against THIS lane's number (local runs with
92            # obsreport installed measure a little higher).
93            # Floor calibrated 2026-06-11 against this lane's measured 72% (local
94            # with obsreport: 73%). Ratchet it UP as coverage grows; never down
95            # without an owner decision.
96            - name: Run full suite (with coverage floor)
97              run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
98
```

14. user (2026-06-16T02:28:23.338Z)

```
1       {
2         "name": "heuristic_velocity_windowing_LOVO",
3         "description": "The honest cross-video floor the Gen-0 learned model must beat:
calibrated velocity-windowing heuristic, leave-one-video-out over the n=6 owned golden corpus.",
4         "computed": "2026-06-10, reproduced via: python -m backend.eval.calibrate_local
--manifest output/human_lovo_manifest.json --metric f1 (matches docs/archives/quality-
iterations/2026-06-multivideo-lovo)",
5         "mean_f1": 0.480,
6         "std": 0.242,
7         "per_video": {
8           "adarsh_avi_singles": 0.751,
9           "kushagra_singles": 0.157,
10          "largetest_doubles": 0.737,
11          "gbaaddy": 0.309,
12          "testlarge_short": 0.267,
13          "mahadevpura_singles": 0.657
14        }
15      }
16
```

15. assistant (2026-06-16T02:28:27.150Z)

Now I have a strong grounding. Let me check the rally_gate and the windowing constants config, plus how `extract_video_chunk`/provider seam is stubbable, to nail down where exactly to freeze the snapshot and how the AI/provider boundary is handled.

16. user (2026-06-16T02:28:28.666Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
15      - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                            detection_params extras such as weights_path).
17
18      This base is deliberately NOT registered; only concrete subclasses register.
19      """
20      import os
21      import time
22      import logging
23      import concurrent.futures
24      from typing import Any, Dict, List, Optional, Tuple
25
26      import cv2
27
28      from backend.pipeline.segmenters.base import BasePlaySegmenter
29      from backend.utils.video import get_video_duration, extract_video_chunk
30      from backend.sports import sport_registry
31      from backend.providers import provider_registry
32      from backend.pipeline.detectors.base import DetectorRunner
33
34      logger = logging.getLogger(__name__)
35
36
37      def _fmt_ts(seconds: float) -> str:
38          seconds = max(0, int(seconds))
39          h, rem = divmod(seconds, 3600)
40          m, s = divmod(rem, 60)
41          return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44      class TrajectoryHybridSegmenter(BasePlaySegmenter):
45          """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""
46
47          #: config section under `indexing` (velocity_threshold lives there), the
48          #: output subdir for trajectories, and the metadata detector-tag prefix.
49          family: str = ""
50          #: human-readable label used in log lines.
51          display_label: str = ""
52
53          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
54              """Return (runner, detector-specific detection_params extras)."""
55              raise NotImplementedError
56
57          @staticmethod
```

```
        def _probe_fps_width(video_path: str) -> tuple:
            """Return (fps, frame_width) for a video, with sensible fallbacks."""
            cap = cv2.VideoCapture(video_path)
            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
            cap.release()
            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

        @staticmethod
        def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
            """Provider-reported exchange count, else a duration-based estimate.

            One canonical implementation ? the twins had drifted (wasb relied on
            ``int(None)`` raising into the fallback; same result, by accident).
            """
            shot_count = segment.get("shuttle_exchange_count")
            if shot_count is None:
                return max(3, int(duration / 0.8))
            try:
                return int(shot_count)
            except (ValueError, TypeError):
                return max(3, int(duration / 0.8))

        # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
            returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
            rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
            features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
            return {
                "peak_velocity": cw["peak_velocity"],
                "mean_velocity": cw["mean_velocity"],
                "active_frame_count": cw["active_frame_count"],
                "track_id_count": cw["track_id_count"],
            }

        def _select_for_handoff(self, windows: List[Dict[str, Any]],
                                window_stats: List[Dict[str, Any]],
                                idx_cfg: Dict[str, Any]) -> List[int]:
            """Indices of the candidate windows that proceed to the AI handoff (and thus may
            become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
            ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
            Persisted candidates are NOT affected ? they remain the full superset for
offline
            re-scoring."""
            return list(range(len(windows)))

        def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                          player_pool: Optional[List[str]] = None,
                          max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
            # override-ignore: the ABC declares the Result contract (Success|Failure)
            # but every live segmenter still returns a bare list ? the documented
            # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
            label = self.display_label
            # --- Sport profile + AI provider wiring (identical across segmenters) ---
            sport_name = self.config.get("sport", "badminton")
            try:
                sport_profile = sport_registry.get(sport_name)
            except KeyError as e:
```

```
118                 logger.error(str(e))
119                 return []
120
```

17. user (2026-06-16T02:28:29.163Z)

```
1       """Segmentation-aware metrics (course-correction C4) ? make over-segmentation visible.
2
3       Temporal IoU (``metrics.score``) is nearly blind to *fragmentation*: a single rally
4       chopped into three predicted pieces, or two rallies merged across a dead-time gap, can
5       still post a moderate mean-IoU. The temporal-action-segmentation / -localization
6       literature grades those failure modes with **F1@k** (segmental F1 at several overlap
7       thresholds) and **mAP@tIoU** (ranked, confidence-aware average precision over a tIoU
8       sweep). Reporting these alongside temporal-IoU is what lets a reconciler's split/merge/
9       tighten win actually show up ? and stops an IoU-chasing rule from shredding rally
10      structure invisibly. See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C4.
11
12      This is **additive**: new functions over the existing ``metrics`` matcher; nothing here
13      changes ``score``/``compare``/LOVO behaviour. Pure (stdlib only), deterministic, no I/O
?
14      so it runs in the GPU-free / numpy-free lanes too.
15
16      Conventions match ``metrics``: an ``Interval`` is a ``(start, end)`` tuple of seconds.
17      A ``ScoredInterval`` is ``(interval, confidence)`` ? the confidence is whatever the
scorer
18      emits per predicted rally (e.g. the classifier probability), used only to *rank* for AP.
19
20      Note on the segmental *edit* score: the classic TAS edit/Levenshtein score operates on
the
21      ordered sequence of segment *labels* and is degenerate for a single action class (every
22      rally carries the same label), so it is intentionally omitted ? F1@k + mAP@tIoU + the
23      segment-count ratio are the over-segmentation-sensitive metrics for this binary setting.
24      """
25      from __future__ import annotations
26
27      from typing import Dict, List, Sequence, Tuple
28
29      from backend.eval.metrics import Interval, interval_iou, score
30
31      #: A predicted interval plus the confidence used to rank it for average precision.
32      ScoredInterval = Tuple[Interval, float]
33
34      #: Default overlap thresholds for F1@k ? the TAS convention F1@{10,25,50}.
35      DEFAULT_OVERLAPS: Tuple[float, ...] = (0.1, 0.25, 0.5)
36      #: Default tIoU sweep for mean average precision ? the action-detection convention.
37      DEFAULT_TIOUS: Tuple[float, ...] = (0.3, 0.4, 0.5, 0.6, 0.7)
38
39
40      def f1_at_overlaps(preds: List[Interval], gts: List[Interval],
41                         overlaps: Sequence[float] = DEFAULT_OVERLAPS) -> Dict[float, float]:
42          """Segmental F1 at each overlap threshold (``{overlap: f1}``) ? the TAS F1@k.
43
44          Reuses the canonical greedy one-to-one matcher, so a fragmented prediction (3 pieces
45          for 1 rally) scores extra false positives and a merged prediction misses a rally ?
46          both drag F1 down where temporal-IoU alone would not flinch.
47          """
48          return {ov: score(preds, gts, iou_threshold=ov).f1 for ov in overlaps}
49
50
51      def average_precision(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
52                            iou_threshold: float = 0.5) -> float:
53          """Average precision at a single tIoU (all-points / VOC-2010 interpolation).
54
55          Predictions are ranked by descending confidence; each is a true positive iff it
56          claims a still-unmatched ground-truth rally at IoU >= ``iou_threshold`` (highest
```

```
        available IoU wins, one GT per prediction). Returns AP in ``[0, 1]``. ``0.0`` when
        there are no ground-truth rallies or no predictions reach the threshold.
        """
        n_gt = len(gts)
        if n_gt == 0 or not scored_preds:
            return 0.0

        order = sorted(range(len(scored_preds)),
                       key=lambda i: (-scored_preds[i][1], i))   # conf desc, stable by
index
        matched_gt = set()
        precisions: List[float] = []
        recalls: List[float] = []
        cum_tp = 0
        cum_fp = 0
        for i in order:
            interval, _conf = scored_preds[i]
            best_iou, best_gi = 0.0, -1
            for gi, g in enumerate(gts):
                if gi in matched_gt:
                    continue
                iou = interval_iou(interval, g)
                if iou >= iou_threshold and iou > best_iou:
                    best_iou, best_gi = iou, gi
            if best_gi >= 0:
                matched_gt.add(best_gi)
                cum_tp += 1
            else:
                cum_fp += 1
            precisions.append(cum_tp / (cum_tp + cum_fp))
            recalls.append(cum_tp / n_gt)

        # Monotone precision envelope (take the max precision to the right), then integrate
        # over recall steps: AP = sum_k (recall_k - recall_{k-1}) * precision_env_k.
        for i in range(len(precisions) - 2, -1, -1):
            precisions[i] = max(precisions[i], precisions[i + 1])
        ap = 0.0
        prev_recall = 0.0
        for p, r in zip(precisions, recalls):
            if r > prev_recall:
                ap += p * (r - prev_recall)
                prev_recall = r
        return ap


    def mean_average_precision(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
                               iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> float:
        """Mean of ``average_precision`` over a tIoU sweep (the standard mAP@tIoU)."""
        tious = list(iou_thresholds)
        if not tious:
            return 0.0
        return sum(average_precision(scored_preds, gts, t) for t in tious) / len(tious)


    def segment_count_ratio(preds: List[Interval], gts: List[Interval]) -> float:
        """Predicted-to-GT segment-count ratio ? a crude fragmentation diagnostic.

        ``> 1`` hints over-segmentation (more predicted pieces than rallies), ``< 1`` hints
        merging/misses. Only meaningful with ground truth present; returns ``0.0`` if
``gts``
        is empty (use F1@k / mAP for the graded picture).
        """
        if not gts:
            return 0.0
        return len(preds) / len(gts)
```

```
120
121
122     def segmentation_report(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
123                             overlaps: Sequence[float] = DEFAULT_OVERLAPS,
124                             iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> Dict[str,
object]:
125         """One-call bundle: F1@k + mAP@tIoU + per-tIoU AP + count ratio.
126
127         ``scored_preds`` carry confidences for AP; F1@k uses the bare intervals. Returns a
128         plain dict (JSON-friendly, string keys) so it can ride next to a ``compare`` result
129         or land in the experiment leaderboard.
130         """
131         preds = [iv for iv, _conf in scored_preds]
132         return {
133             "f1_at_overlaps": {f"{ov:g}": v for ov, v in f1_at_overlaps(preds, gts,
overlaps).items()},
134             "ap_at_tiou": {f"{t:g}": average_precision(scored_preds, gts, t) for t in
iou_thresholds},
135             "mean_ap": mean_average_precision(scored_preds, gts, iou_thresholds),
136             "segment_count_ratio": segment_count_ratio(preds, gts),
137             "num_pred": len(preds),
138             "num_gt": len(gts),
139         }
140
```

18. assistant (2026-06-16T02:28:32.896Z)

I now have everything I need. I have a complete picture of the seam, the scoring stack, the determinism risks, the fixture conventions, the CI lanes, and the licensing/privacy guardrails. Let me compose the design and return it via StructuredOutput.

19. user (2026-06-16T02:30:37.613Z)

Structured output provided successfully

20. assistant (2026-06-16T02:30:49.666Z)

I've returned the full design via StructuredOutput. Key load-bearing facts from the codebase that shaped it:

- The freeze seam is the trajectory CSV (`Frame,Visibility,X,Y`) ? `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\tracknet_runner.py:237` (`parse_trajectory_csv`) and `:256` (`trajectory_to_action_windows`). Everything downstream is pure stdlib float math; `fps`/`frame_width` MUST be pinned because `_probe_fps_width` (`backend/pipeline/segmenters/trajectory_hybrid.py:58`) silently falls back to 30.0/1280.0.
- The scoring stack is reused verbatim: `backend/eval/metrics.py::score`/`match_intervals` (deterministic greedy, empty?0.0) and `backend/eval/segmentation_metrics.py` (f1@k, segment_count_ratio).
- The replay-from-cached-CSV pattern already exists in `backend/eval/calibrate_wasb.py::make_predict_fn` and is exercised on real trajectories in `tests/test_served_gate_a_regression.py` ? which currently skips in CI because the corpus is uncommitted. Committing tiny fixtures is exactly the gap that makes it CI-runnable.
- Provenance/incommensurability discipline is borrowed from `backend/eval/regression.py` (`gt_hash`, the IoU/detector/gt_hash guards, exit 0/1/3) and the `eval_baselines/heuristic_lovo_n6.json` committed-artifact convention.
- CI lands in the existing `full-suite` lane (`.github/workflows/ci.yml`, 70% floor); no new job, no GPU.

I flagged the two honest tensions explicitly in the structured fields: the false-safety boundary (detector, video-IO, provider, reel bytes, and untested branches are NOT covered ? this is characterization, not correctness), and that real-derived fixtures reverse the current `GOLDEN_DATA_SHARING.md` "keep proprietary-derived data out of git" convention, making synthetic-first the recommended, owner-gate-free default.